

**THE UNIVERSITY OF DANANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF MECHANICAL ENGINEERING**



**PBL 3: DESIGNING DEVICE APPLYING
MICROCONTROLLERS AND SENSORS**

23.05A

**TOPIC: STATIC DRIVING CABIN
SIMULATOR**

SUPERVISOR: Assoc. Prof. Dr. DANG PHUOC VINH

STUDENT: DANG QUOC CANH 101230277

BUI KHAC BAO TRI 101230465

NGO TRAN VINH QUANG 101230305

GROUP: 10

Da Nang, June/2026

PREPACE

Nowadays, transportation plays a very important role in social development and human life. In which, automobiles are an essential means of transport in all aspects. Along with that importance, the rapid increase in vehicles brings serious issues regarding traffic accidents, and real-world driving training inherently carries many risks. Therefore, applying modern technology to create a safe driver training environment becomes a top priority.

One of the most advanced technologies widely used today is virtual simulation. And the system that is widely encouraged to be used by humans to safely recreate a real driving environment is the car driving simulator.

However, if we only use simple simulation software on a computer without interactive hardware, learners will not experience the most realistic driving sensation. To improve training effectiveness and realism, what solutions do we have to come up with? Using the knowledge learned and practical surveys, our team decided to choose the topic: "*Static Driving Cabin Simulator*".

Under the enthusiastic guidance and instruction of *Mr. Dang Phuoc Vinh*, through the research process, the group completed the hardware model and surveyed the operation of the simulator. However, due to time constraints, as well as limited knowledge, we cannot avoid shortcomings. Therefore, we look forward to receiving many comments and suggestions from teachers to be able to develop and improve this topic further.

We sincerely thank you!

Dang Quoc Canh
Bui Khac Bao Tri
Ngo Tran Vinh Quang

CONTENTS

PREPACE.....	2
CONTENTS	3
CHAPTER 1: INTRODUCTION	5
1.1. Purpose and requirements of the project	5
1.2. Introduction about Static Driving Cabin Simulator.....	5
1.3. Reasons to use the Static Driving Cabin Simulator.....	6
1.4. Advantages and Disadvantages of the Static Driving Cabin Simulator	6
1.5. Applications of the Static Driving Cabin Simulator.....	7
1.6. Structure of the Static Driving Cabin Simulator	8
CHAPTER 2: HARDWARE DESIGN	10
2.1. Proteus 8 Professional.....	11
2.2. PIC16F877A	12
2.3. DC Motor.....	14
2.4. Ultrasonic Sensor HY-SRF05.....	15
2.5. LCD 16x2	16
2.6. Module I2C	17
2.7. L298N Dual H-Bridge Motor Driver.....	19
2.8. Rotary Potentiometer	20
2.9. Module Encoder.....	21
2.10. Servo motor MG90S	22
2.11. Module PCA9685.....	24
2.12. Module RS232.....	25
2.13. Hardware Design.....	26
2.13.1. Power Supply	26
2.13.2. Controller circuit	28
CHAPTER 3: SOFTWARE DESIGN	33
3.1. CCS Compiler.....	33
3.2. PICKIT 3 v3.10.....	34
3.3. Algorithm Flowchart	35
3.3.1. Main program	35

3.3.2. Function program	42
3.3.3. External Interrupt program.....	44
3.4. Code CCS Compiler program.....	44
3.5. Visual Studio	54
3.6. Visual Studio program	55
CHAPTER 4: CONCLUSION	62
4.1. Results.....	62
4.2. Problems	62
4.3. Future advantage.....	62
REFERENCES	63

CHAPTER 1: INTRODUCTION

1.1. Purpose and requirements of the project

Purpose:

Researching, designing, and manufacturing a car driving simulator device aims to create a safe and cost-effective practice environment. It helps users familiarize themselves with driving operations (steering, accelerating, braking, and clutching) in a realistic virtual environment without incurring fuel costs or vehicle depreciation, while completely eliminating the risk of traffic accidents. Furthermore, the project comprehensively applies the acquired knowledge of microcontrollers, sensor technology, electronic circuit design, and mechanical engineering to develop a functional hardware product.

Requirements of the project:

- Research the structure and operating principles of actuators in a real automotive cockpit.
- Design control circuits for sensors, DC motors, etc.
- Write the control program for the PIC16F877A microcontroller.
- Simulate operational mechanisms such as the steering wheel, accelerator, brake, windshield wipers, headlight system, turn signals, etc.

1.2. Introduction about Static Driving Cabin Simulator



Fig 1.1: Static Driving Cabin Simulator

The static driving cabin simulation system is designed as a visual physical hardware model, rather than utilizing virtual reality simulation software on a computer. The core objective of the model is to realistically recreate the basic electrical operations and actuators of an automobile through a microcontroller and electronic components.

Structurally, the system is a tight integration of three main modules: the input module (comprising the steering wheel, accelerator pedal, brake pedal, and control switches for the windshield wipers, lighting system, and turn signals); the central processing unit utilizing the PIC16F877A microcontroller, which acts as the "brain"

to collect data and process logic; and the output module, which includes DC motors simulating the movement of the wheels and windshield wipers, a physical lighting system, combined with safety warning systems such as backup sensors and mirror sensors.

Operationally, when the user physically interacts with the input mechanisms (such as pressing the accelerator, steering, or turning on a switch), the microcontroller reads the voltage signals, calculates, and directly controls the corresponding physical actuators (for example: changing the rotational speed of the wheels using PWM techniques, turning on the lighting system, or activating an indicator light when a sensor detects an obstacle). The model will provide a dynamic experimental platform, helping students and users easily observe and grasp the operating principles of automotive mechatronic systems in the most intuitive and safe manner.

1.3. Reasons to use the Static Driving Cabin Simulator

Maximizing experimental and learning efficiency: By directly replicating the electrical and mechanical mechanisms of a real car, the physical hardware simulation model allows users to intuitively observe and understand the real-time operation of actuators (like DC motors and lighting systems) and sensors. This hands-on approach ensures maximum absorption of practical mechatronic knowledge.

Increased economic and safety efficiency: By utilizing a hardware simulator, educational and testing processes can be conducted without the need for a real, expensive vehicle. This significantly reduces training and research costs by saving on fuel, maintenance, and vehicle depreciation. Additionally, using a simulator completely eliminates the risks of real-world traffic accidents, ensuring a perfectly safe environment for students and users.

Flexibility and responsiveness: The microcontroller-based simulation system can be easily reprogrammed to adjust the parameters of sensors and actuators according to different testing needs. This enables the model to adapt quickly to various simulated scenarios, such as changing the obstacle detection range of the backup sensors or modifying the PWM speed of the wheel motors under different driving conditions.

1.4. Advantages and Disadvantages of the Static Driving Cabin Simulator

Advantages

High visual and practical value: Utilizing actual hardware components to simulate the steering wheel, pedals, sensors, and DC motors allows users and students to easily observe, measure, and understand the operating principles of automotive actuators.

Low manufacturing cost and high accessibility: Compared to virtual reality simulation systems that utilize 3D screens and 6-degree-of-freedom (6-DOF) hydraulic platforms, this model employs a widely available microcontroller

(PIC16F877A) and basic electronic components, thereby optimizing research costs for students.

Ensuring safety: The system provides a practical environment for beginners to understand basic driving controls, completely eliminating the risks of collisions or traffic accidents associated with practicing on a real vehicle.

Flexible customizability: As the system is built on a microcontroller platform, developers can easily reprogram the code to modify operational parameters (e.g., changing the PWM speed range of the wheels or adjusting the backup sensor's alarm distance) to meet specific testing requirements.

Disadvantages

Lack of a virtual simulation environment: The most significant limitation of the current model is that it is not yet connected to computer software to display 3D graphics. Consequently, users do not experience a realistic visual perception of the vehicle moving on the road, environmental factors, or forward obstacles.

Lack of Force Feedback systems: The current steering wheel and pedal assemblies primarily rely on potentiometer mechanisms for simulation.

Processing limits of the 8-bit microcontroller: The PIC16F877A (8-bit) microcontroller handles current basic logic tasks exceptionally well. However, if the project is to be expanded with more complex features such as PC communication, rearview camera image processing, central display control, or mobile app control, the chip's memory and processing speed will become a significant bottleneck.

1.5. Applications of the Static Driving Cabin Simulator

Car cockpit simulation (virtual cabin) is widely applied in driving training, safety reflex conditioning, new vehicle research and development, and entertainment. It helps learners familiarize themselves with vehicles and practice skills under complex weather and traffic conditions without risks, while simultaneously saving on fuel and maintenance costs.

The main applications include:

- **Driving training and examination:** 3D simulation cabins help new learners get accustomed to control components (steering wheel, gear shifter, accelerator/brake pedals) in a safe environment. The system allows for practicing real-world scenarios such as city driving, highway driving, and parking.
- **Safety skill conditioning:** Drivers are trained to handle dangerous and unexpected situations (pedestrians crossing the street, collisions, bad weather) to improve their reflexes without causing any actual danger.
- **Specialized vehicle training:** Applied in simulating specific types of vehicles such as taxis, police cars, ambulances, and heavy-duty trucks.

- Automotive Research and Development (R&D): Car manufacturers utilize 6-DOF (Six Degrees of Freedom) virtual cockpits to test designs, Advanced Driver Assistance Systems (ADAS), and driver perception prior to actual mass production.
- Entertainment and racing simulation: Simulation applications and games on PCs and mobile devices (such as City Car Driving, Real Car Driving) allow users to experience realistic driving sensations, drifting, or racing.



Fig 1.2: 6-DOF Car Racing Simulator

1.6. Structure of the Static Driving Cabin Simulator

The car cockpit simulation model is designed based on the principles of a standard mechatronic system, involving the close coordination of mechanics, electronics, and control programming. The overall structure of the system is divided into four main functional blocks:

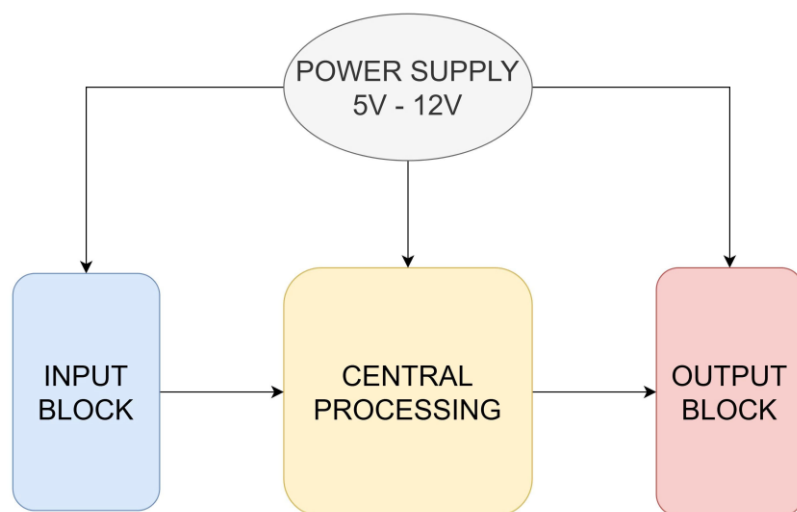


Fig 1.3: Structure of the system

- Input Block: Responsible for collecting signals from the operator and the environment. This block includes physical interface mechanisms such as the

steering wheel, accelerator/brake pedal assembly (using potentiometers to generate analog ADC signals), control switches (wipers, turn signals, headlights), and sensor systems (ultrasonic/infrared sensors used for reversing and blind-spot warnings).

- Central Processing Block: Acts as the "brain" of the entire system, utilizing the PIC16F877A microcontroller. This block is tasked with receiving signals from the input block, processing the pre-programmed logic algorithms, and outputting corresponding control signals (such as Digital signals or PWM).
- Output & Actuator Block: Receives signals from the microcontroller to execute physical actions. This block consists of power modules (Motor drivers, Relays), DC motors (simulating the rotation of wheels and wipers), LED/lighting systems, and a buzzer to emit warning sounds.
- Power Supply Block: Ensures a stable energy supply for the entire system's operation. This block converts and distributes different voltage levels, typically 5VDC for the microcontroller and sensors, and 12VDC/5VDC for the motors and lighting systems.

CHAPTER 2: HARDWARE DESIGN

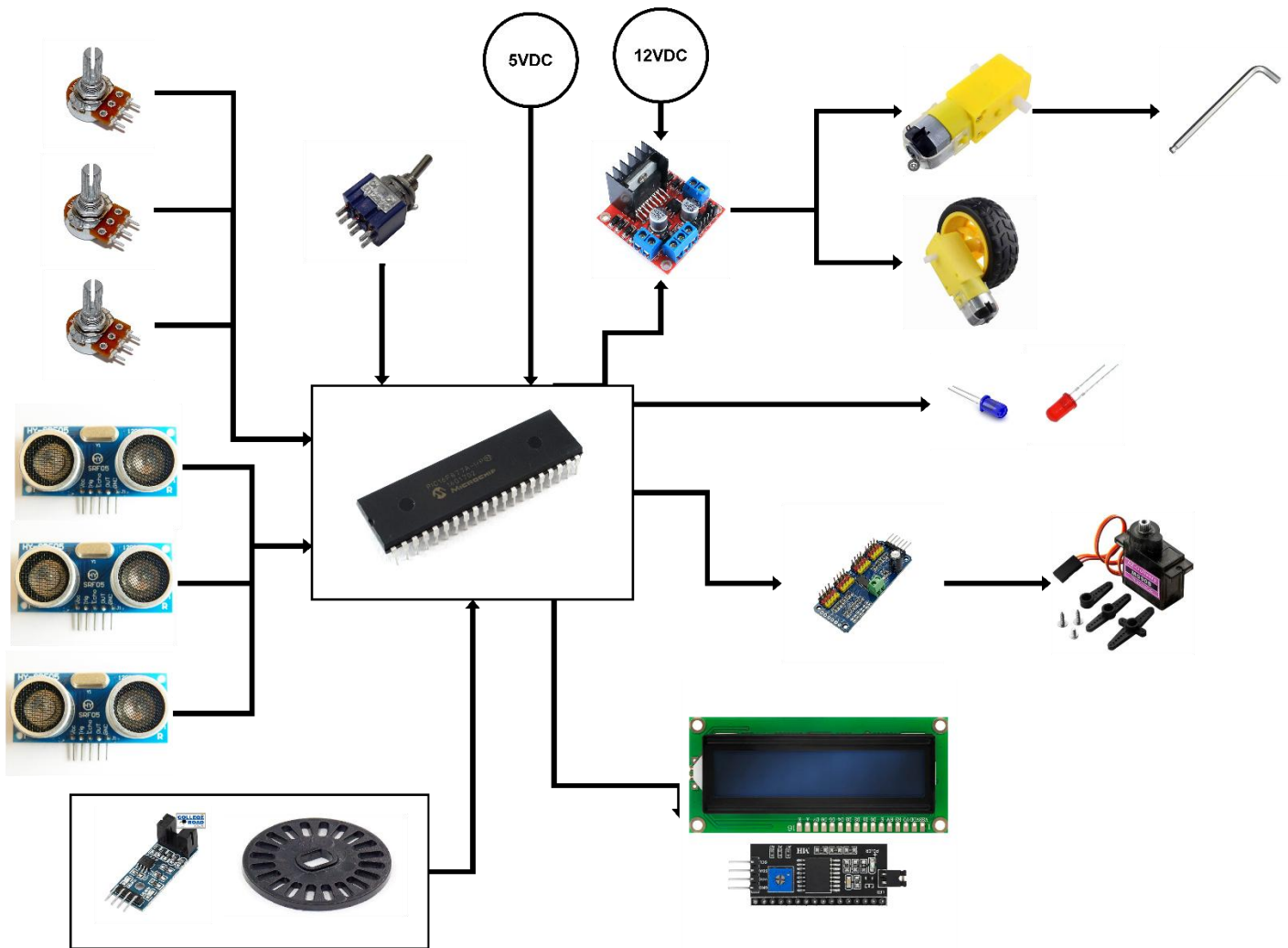


Fig 2.1: Car Cabin Simulation System Overview Diagram

This Static Driving Cabin Simulator utilizes the PIC16F877A microcontroller as the central processing unit (ECU) to collect data and synchronize all operations. The driver interacts directly via three potentiometers (simulating the steering wheel, accelerator, and brake pedals) and a toggle switch (simulating the gear shifter). These control signals are transmitted through an L298N motor driver module to operate the DC motor for forward and reverse movement, and a PCA9685 module to control a servo motor for left and right steering, combined with an encoder to calculate the actual simulated speed. To enhance realism, the system integrates an Advanced Driver Assistance System (ADAS) feature with three ultrasonic sensors continuously scanning the blind spots and the rear of the vehicle, thereby automatically activating a cluster of three LEDs for brake indication or collision warnings on both side mirrors. Operating parameters and the simulated vehicle speed are displayed on an LCD screen.

2.1. Proteus 8 Professional



Proteus Professional is a software tool used for electronic circuit design and simulation. It is widely utilized by engineers, students, and hobbyists for developing and testing circuits before they are physically implemented. Proteus8 Professional, often simply referred to as Proteus, is one of the latest versions of the software, offering advanced features and capabilities.

Here is a brief introduction to Proteus8 Professional:

- **Electronic Circuit Design:** Proteus allows users to design electronic circuits using a user-friendly schematic capture interface. You can create circuits by placing and connecting components such as resistors, capacitors, transistors, and ICs.
- **Simulation:** One of the key features of Proteus is its powerful simulation engine. Users can simulate the behavior of their circuits before building a physical prototype. This helps in identifying and rectifying potential issues and errors in the design.
- **Mixed-Mode Simulation:** Proteus supports mixed-mode simulation, allowing the simulation of both analog and digital components in the same circuit. This is particularly useful for projects that involve both types of components.
- **Microcontroller Simulation:** Proteus includes a wide range of microcontroller models, enabling users to simulate and test embedded systems. This feature is valuable for those working on projects involving microcontroller-based designs.
- **PCB Layout Design:** After designing and simulating the circuit, users can proceed to the layout design phase. Proteus provides tools for creating professional-looking printed circuit board (PCB) layouts. Users can place components, route traces, and generate manufacturing files.

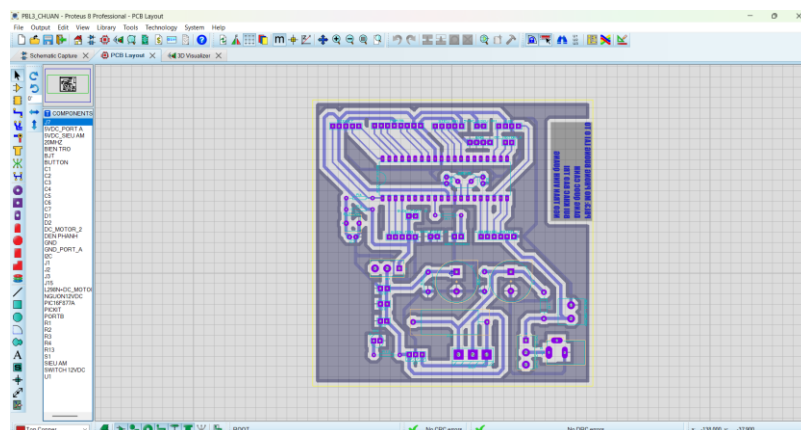


Fig 2.2: Proteus PCB layout

- **Real-Time Interaction:** Proteus supports real-time interaction with the circuit during simulation. Users can observe the behavior of the circuit and monitor various parameters in real-time, providing valuable insights into the design's performance.
- **3D Visualization:** Proteus8 Professional enhances the user experience with 3D visualization of the PCB layout. This feature allows users to view the PCB in a three-dimensional space, aiding in the identification of design issues and improving the overall design process.
- **Libraries and Components:** The software comes with an extensive library of components and models, covering a wide range of electronic devices. Users can also create custom components to meet specific project requirements.

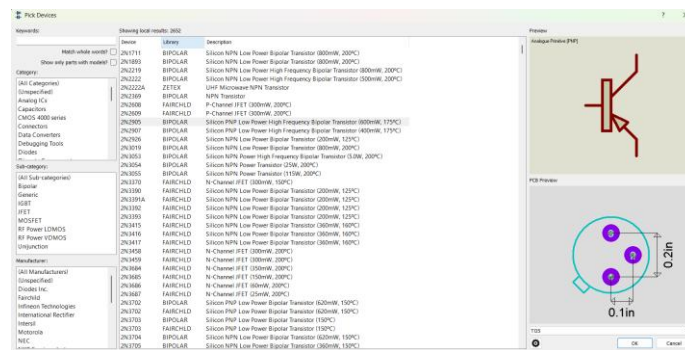


Fig 2.3: Proteus's library

- **Educational Use:** Proteus is widely used in educational institutions for teaching electronic circuit design and simulation. It provides a hands-on approach to learning and allows students to gain practical experience in a virtual environment.

Overall, Proteus8 Professional is a comprehensive tool that facilitates the entire electronic design process, from schematic capture to simulation and PCB layout. It is an invaluable resource for professionals, students, and hobbyists involved in electronics and circuit design.

2.2. PIC16F877A

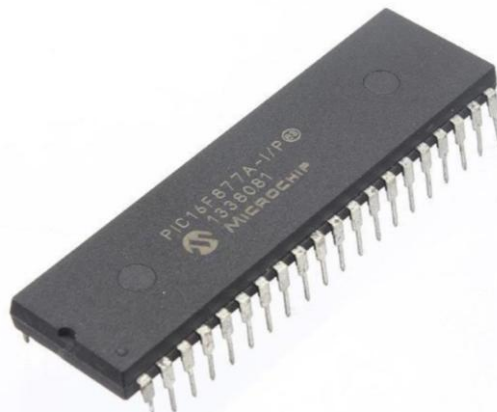


Fig 2.4: [PIC16F877A](#)

The PIC16F877A is an 8-bit microcontroller developed by Microchip Technology. It is one of the popular and widely used 8-bit microcontrollers in embedded and electronic applications.

Here are the detailed specifications of the PIC16F877A:

- Architecture: The microcontroller is based on the RISC (Reduced Instruction Set Computing) architecture, featuring a simple and easy-to-understand instruction set. It has a powerful CPU with a processing speed of up to 5 MIPS (Million Instructions Per Second).
- Memory: The PIC16F877A has 14KB of Flash memory for storing user programs. The Flash memory is reprogrammable, allowing for easy updates and modifications. Additionally, it has 368 bytes of RAM for temporary data storage during program execution.
- Communication: The microcontroller supports various communication interfaces such as USART (Universal Synchronous/Asynchronous Receiver/Transmitter), SPI (Serial Peripheral Interface), and I2C (Inter-Integrated Circuit). This enables seamless communication with external peripherals such as sensors, displays, and other devices.
- Interrupts: The PIC16F877A offers 14 interrupt sources, allowing the microcontroller to respond quickly and handle important events as they occur. Interrupts provide real-time capabilities to the applications.
- Power Control: The microcontroller features power-saving modes and supports multiple power-saving levels. These power-saving modes help extend battery life and reduce power consumption.
- I/O Pins: The PIC16F877A provides 33 I/O (Input/Output) pins for connecting and controlling external peripherals and sensors. The I/O pins support both input and output operations, providing flexibility in interacting with the external world.
- PIC16F877A is a 40-pin PIC Microcontroller and is used mostly in embedded projects and applications. It has five ports starting from port A to port E. It has three timers of which two are 8-bit timers and one is 16-bit timers. It supports many communication protocols such as serial protocol, parallel protocol, I2C protocol. The PIC16F877A supports both hardware pin interrupts and timer interrupts.
- Flash program memory with capacity of 8K x 14 bits, can be read/write up to 100.000 times
- RAM Memory size of 368 byte
- EEPROM Memory size of 256 bytes. Can be read and written up to 1.000.000 times. Data can be stored up to 40 years Supply power: 5VDC
- Sleep mode for saving power usage
- Five I/O port with 33 pins, namely A, B, C, D, E.

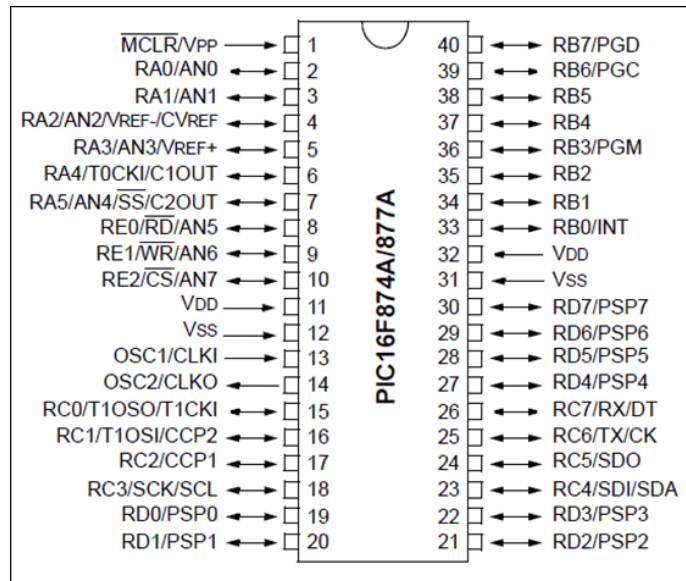


Fig 2.5: PIC16F877A pinout

Function of the pins:

MCLR/Vpp (1): Reset input, low active

PORT A: 6 pins (2-7) can act as both input/output of PIC

With analog input of ADC

Comparator voltage input

Input of external clock of Timer 0 (RA4)

MSSP (Master Synchronous Serial Port) interface input

PORT B: 8 pins (33-40) can act as both input/output of PIC

Built-in internal pull-up resistor

PORT C: 8 pins (15-18, 23-26) can act as both input/output of PIC

Integrated serial communicators I2C, SPI, SSP, USAT

Input/output of external clock of timer 1

PORT D: 7 pins (19-20, 21-22, 27-30) can act as both input/output of PIC

PORT E: 4 pins (8-10,) can act as both input/output of PIC

VDD,VSS: pins (11-12,31-32) Power supply pins

OSC1/CLKIN, OSC2/CLKOUT pins (13,14) Connecting to crystal device to create oscillation for PIC.

2.3. DC Motor



Fig 2.6: DC motor

A DC motor (Direct Current motor) is a common electromechanical device that converts electrical energy (direct current) into mechanical energy (rotational motion). This device is widely used in machinery systems that require flexible speed control and reversible rotation. Here is an overview of DC motors:

- **Construction:** A basic DC motor (brushed type) consists of two main parts: the Stator (the stationary part – usually a permanent magnet or electromagnet used to generate a magnetic field) and the Rotor (the moving part – the armature carrying coils wound around a steel core). In addition, the motor includes a commutator and brushes, which serve to supply power and continuously reverse the direction of the current flowing into the rotor coils.
- **Operating Principle:** DC motors operate based on the principle of electromagnetic force (Lorentz force). When a direct current flows through the rotor coil placed within the stator's magnetic field, the interaction between the external magnetic field and the current generates a torque, causing the motor shaft to rotate. The rotational speed of the shaft is proportional to the average voltage supplied to the motor terminals.
- **Control Interface:** Because microcontrollers cannot provide sufficient current to directly drive the load, DC motors are typically controlled through auxiliary power circuits. The speed of the motor is controlled using Pulse Width Modulation (PWM) techniques. To reverse the direction of rotation, the system usually utilizes an H-bridge circuit, such as the L298N motor driver module, which receives logic signals from the microcontroller to easily change the direction of the current flowing through the motor.

2.4. Ultrasonic Sensor HY-SRF05

The HY-SRF05 Ultrasonic Sensor is used to determine the distance from an object to the sensor using ultrasonic waves. The sensor features a fast response time and high precision, making it highly suitable for applications such as obstacle detection and ultrasonic distance measurement.

The HY-SRF05 ultrasonic sensor offers two operating modes: using a separate pair of Echo / Trigger pins or using a single OUT pin to both transmit and receive signals.



Fig 2.7: Ultrasonic Sensor HY-SRF05

Technical Specifications:

- Operating Voltage: 5VDC
- Operating Current: 10~40mA
- Communication Signal: TTL logic level
- Signal Pins: Echo, Trigger (commonly used), and Out (rarely used).
- Detection Angle: < 15 degrees
- Operating Frequency: 40kHz
- Measuring Range: 2~450cm (The maximum distance is achieved under ideal conditions with open space and flat object surfaces. Under normal conditions, the sensor provides the most accurate results at distances < 100cm).
- The basic formula to calculate the distance is:

$$Distance = \frac{Velocity \times Time}{2}$$

- Accuracy: ±0.3cm (The closer the distance and the flatter the object's surface, the smaller the error margin).
- Dimensions: 43mm x 20mm x 17mm

Function of the pins:

VCC: Connects to the 5V positive power supply.

Trig (Trigger): The trigger input pin used to emit ultrasonic pulses.

Echo: The receiver output pin that receives the reflected signal to calculate the time of flight.

OUT (or Mode):

If left unconnected: Operates in Mode 1 (similar to the HC-SR04), using separate Trig and Echo pins.

If connected to GND: Operates in Mode 2, allowing a single pin to be used for both transmitting and receiving signals, thereby saving I/O pins on the microcontroller.

GND: Connects to the 0V ground.

2.5. LCD 16x2

LCD1602 text screen using HD44780 driver, capable of displaying 2 lines with 16 characters each, the screen is durable, very popular, lots of sample code and easy to use, suitable for beginners.

The display is easy to communicate and displays parameters according to the ASCII code table. It is also possible to add information display with custom characters.

Specifications of LCD1602:

- MAX voltages: 7V
- MIN voltages: - 0.3V
- Stable operation: 2.7-5.5V
- High output voltage: > 2.4

- Low output voltage: <0.4V
- Power supply current: 350uA - 600uA
- Operating temperature: -30 ÷ 75 °C

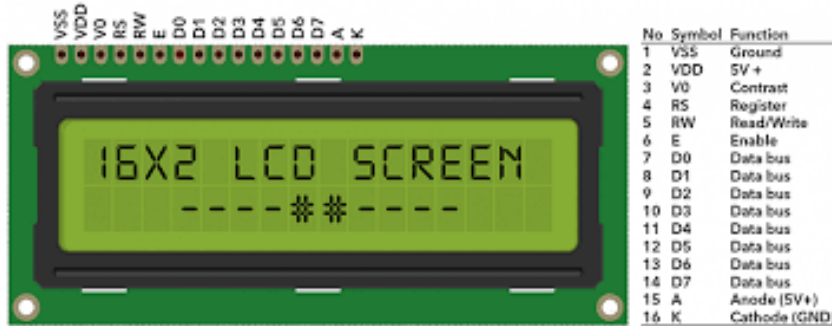


Fig 2.8: LCD16x2 pinout

Function of the pins:

VSS (GND): Power supply pin for LCD, connected to VCC=5V of the control circuit

VEE (V0): Usually connected to a variable resistor. Adjust the contrast of the LCD

RS (Resister Select): Register select pin, connected to logic "0" or logic "1"

RW (Read/Write): Read/write mode selection pin (Read/Write), connected to logic "0" to write or connected to logic "1" read

E (Enable): The pin allows the LCD to work

D0 – D7: The eight lines of the data bus are used to exchange information with the MPU. There are two modes of using these 8 bus lines: 8-bit mode (data is transferred on all 8 lines, with the MSB bit being the DB7 bit) and 4-bit mode (data is transferred over 4 lines from DB4 to DB7, the MSB bit is DB7)

A (Anode): Used to provide positive power to the backlight (5V+)

K (Cathode): Used to provide negative power for the backlight (GND)

2.6. Module I2C

I2C (Inter-Integrated Circuit) is a synchronous serial communication protocol that allows multiple peripheral devices to be connected to the same bus using only two signal wires.

Hardware Structure: The I2C module utilizes two primary signal lines:

- SDA (Serial Data): A bidirectional data transmission line.
- SCL (Serial Clock): A clock signal line used to synchronize data transmission.
- Both lines must be connected to pull-up resistors to keep the signal at a high logic level when no device is transmitting.

Operating Mechanism:

- Master-Slave Mechanism: One device acts as the Master (typically a microcontroller), which generates the clock signal and initiates communication. The remaining devices act as Slaves (sensors, LCD screens, EEPROMs, etc.)

- Device Addressing: Each Slave on the bus has a unique address (usually 7-bit or 10-bit). The Master calls this specific address to begin transmitting or receiving data.



Fig 2.9: Module I2C

The I2C module features a row of 16 output pins that connect directly to the 16 pins of the 16x2 LCD (ensuring that pin 1 of the module corresponds to pin 1 - VSS of the LCD).

PCF8574: The core I/O expander chip responsible for converting I2C serial data from the microcontroller into 8-bit parallel data for the LCD.

I2C Interface: A 4-pin header (GND, VCC, SDA, SCL) used to establish communication with the main microcontroller (like the PIC16F877A).

- GND: Connects to the negative terminal (0V / Ground).
- VCC: Connects to the positive power supply (typically 5V).
- SDA (Serial Data): The data pin (connects to the microcontroller's SDA pin).
- SCL (Serial Clock): The clock pin (connects to the microcontroller's SCL pin).

Potentiometer (Trimpot): Rotate to adjust the contrast of the characters on the screen. If the backlight is on but no text is visible, adjust this component.

Backlight Jumper: Used to turn the backlight on or off. Removing the jumper turns off the backlight.

Device Address Set (A0, A1, A2): Used to change the I2C address if multiple LCDs need to be connected simultaneously. The default address is usually 0x27 or 0x3F.

Power LED: An indicator light that turns on when the module receives power.

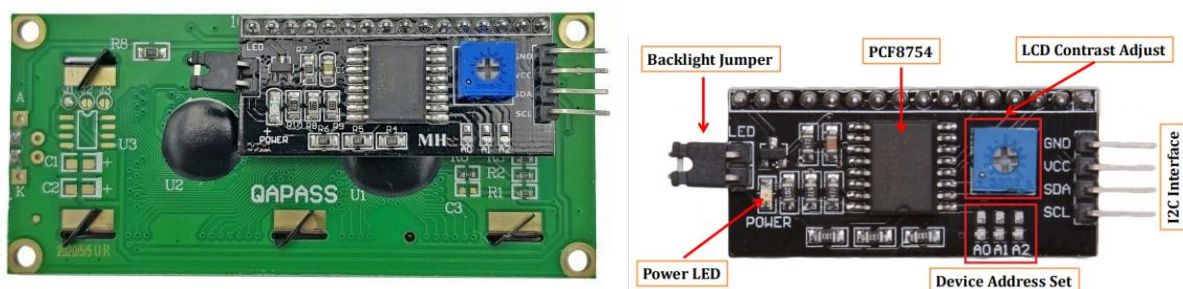


Fig 2.10: I2C Module for 16x2 LCD Display

2.7. L298N Dual H-Bridge Motor Driver

This dual bidirectional motor driver utilizes the highly prevalent L298 Dual H-Bridge Motor Driver IC. The module enables seamless and independent control of either two DC motors or a single stepper motor, supporting up to 2A per channel in both rotational directions. It is ideal for robotic applications and well suited for connection to a microcontroller requiring just a couple of control lines per motor. It can also be interfaced with simple manual switches, TTL logic gates, relays, etc. This board equipped with power LED indicators, on-board +5V regulator and protection diodes.

Technical Specifications:

- Input Voltage: 3.2V ~ 40VDC
- Driver: L298N Dual H Bridge DC Motor Driver
- Power Supply: DC 5 V - 35 V
- Peak current: 2 Amp
- Operating current range: 0 ~ 36mA
- Control signal input voltage range :
 - Low: $-0.3V \leq V_{in} \leq 1.5V$.
 - High: $2.3V \leq V_{in} \leq V_{ss}$.
- Enable signal input voltage range :
 - Low: $-0.3 \leq V_{in} \leq 1.5V$ (control signal is invalid).
 - High: $2.3V \leq V_{in} \leq V_{ss}$ (control signal active).
- Maximum power consumption: 20W (when the temperature $T = 75^{\circ}\text{C}$).
- Storage temperature: $-25^{\circ}\text{C} \sim +130^{\circ}\text{C}$.
- On-board +5V regulated Output supply (supply to controller board i.e. Arduino).
- Size: 3.4cm x 4.3cm x 2.7cm

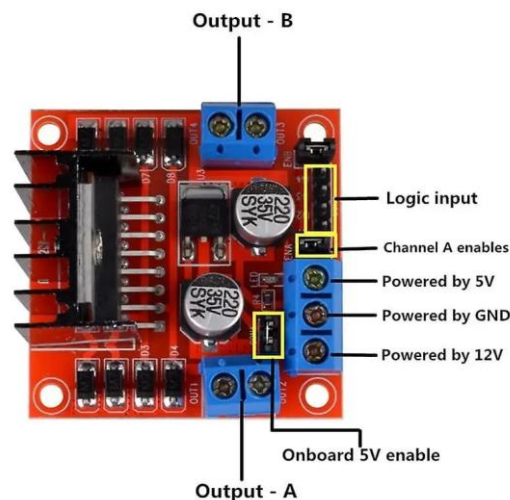


Fig 2.11: L298N Dual H-Bridge Motor Driver

Function of the pins:

VCC (hoặc 12V): Motor power supply input (accepts up to 35V DC).

GND: Ground connection (must be connected to the microcontroller's ground

5V: Logic supply pin. If the 5V jumper is in place, this pin acts as a 5V output to power the microcontroller. If the jumper is removed, a 5V input must be provided here.

IN1 & IN2: Logic input pins to control the direction of Motor A.

IN3 & IN4: Logic input pins to control the direction of Motor B.

ENA (Enable A): PWM input pin to control the speed of Motor A. (If connected with a jumper, the motor runs at maximum speed).

ENB (Enable B): PWM input pin to control the speed of Motor B.

OUT1 & OUT2 (Output A): Output terminals to connect Motor A.

OUT3 & OUT4 (Output B): Output terminals to connect Motor B.

2.8. Rotary Potentiometer



Fig 2.12: Rotary Potentiometer

A Rotary Potentiometer is a three-terminal variable resistor that adjusts resistance through a rotating wiper moving across a circular resistive element. Functioning primarily as a voltage divider, it is widely utilized for controlling electrical devices, with applications ranging from volume adjustment and lighting dimmers to position sensing. These components typically feature a rotational range of 300° and are available in linear or logarithmic tapers.

Key Features and Classifications:

Operating Principle: Rotating the shaft moves the wiper along the resistive track, proportionally altering the voltage at the output terminal.

Pin Configuration: The two outer terminals are connected to the ends of the resistive element, while the center terminal connects to the adjustable wiper.

Linear vs. Logarithmic Tapers: Linear potentiometers vary resistance in direct proportion to the rotation angle. Logarithmic (or audio) potentiometers change resistance logarithmically to match human auditory perception.

Types: Common variants include single-turn (standard applications), multi-turn (high-precision tuning), and wire-wound (high-power handling) models.

Primary Functions: Extensively employed as user control interfaces (e.g., volume knobs) and for providing sensor feedback regarding angular position.

2.9. Module Encoder



Fig 2.13: Module Encoder

The U-shaped Infrared Speed Sensor Encoder Module is an electronic device that utilizes a pair of opposing infrared transmitter and receiver components to measure the rotational speed of a motor or detect obstacles. When a slotted encoder disk passes through the U-shaped slot, the infrared beam is interrupted, generating electrical pulses for a microcontroller to process.

Structure:

Optical Sensor (Photo Interrupter): Commonly uses components like the ITR9608-F, consisting of an infrared emitting LED and a photo-transistor housed in a U-shaped plastic casing.

Comparator IC: Typically utilizes the LM393 IC to stabilize the output pulse signal (level 0 or 1), enabling the microcontroller to read the data more accurately.

Pin Connections:

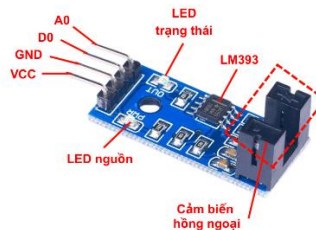


Fig 2.14: Module Encoder pinout

VCC: Power supply (usually 3.3V - 5V).

GND: Ground (Negative pole of the power supply).

DO (Digital Output): TTL digital pulse signal (0 or 5V) outputted when there is an obstacle or a slot.

AO (Analog Output): Analog signal (depending on the module version).

Indicators: Usually includes a power indicator LED and an output signal status LED.

Operating Principle:

Unobstructed state: The infrared beam from the transmitter travels straight to the receiver through the gap. The DO pin is typically at a low (0V) or high (5V) level, depending on the circuit design.

Obstructed state: When the encoder disk (or an opaque object) blocks the infrared beam, the receiver does not receive the signal, and the DO pin toggles its state.

Speed Calculation: A microcontroller (such as an Arduino or PIC) counts the number of pulses (state changes) within a specific time frame to calculate the motor's revolutions per minute (RPM).

Applications:

Robotics: Measuring travel distance and controlling the speed of robot wheels.

Motor speed measurement: Combined with an encoder disk attached to the motor shaft to monitor the actual rotational speed.

Position detection: Determining mechanical stop points in printers or automated systems.

2.10. Servo motor MG90S



Fig 2.15: Servo motor MG90S

A servo motor is a type of electromechanical device that is widely used for precise control of angular or linear position, velocity, and acceleration. It consists of a motor, a control circuit, and a feedback system. Here is an overview of servo motors:

Construction: Servo motors are typically made up of a DC motor, gears, a position or velocity sensor (such as a potentiometer or an encoder), and a control

circuit. The motor drives the gears, which in turn control the output shaft's movement.

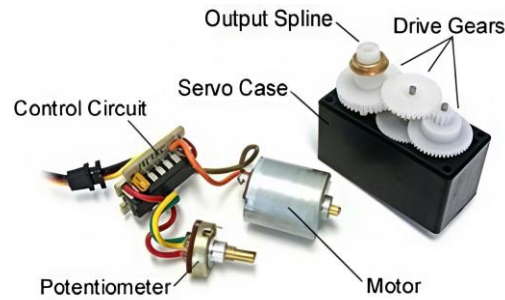


Fig 2.16: Construction servo

Operating Principle: Servo motors operate based on a closed-loop control system. The control circuit receives a control signal, typically in the form of pulse width modulation (PWM) and compares it to the feedback signal from the position or velocity sensor. It adjusts the motor's output to achieve and maintain the desired position or speed.

Control Interface: Servo motors are typically controlled using PWM signals, where the width of the pulse determines the desired position or speed. The control signal is often generated by a microcontroller or a dedicated servo controller.

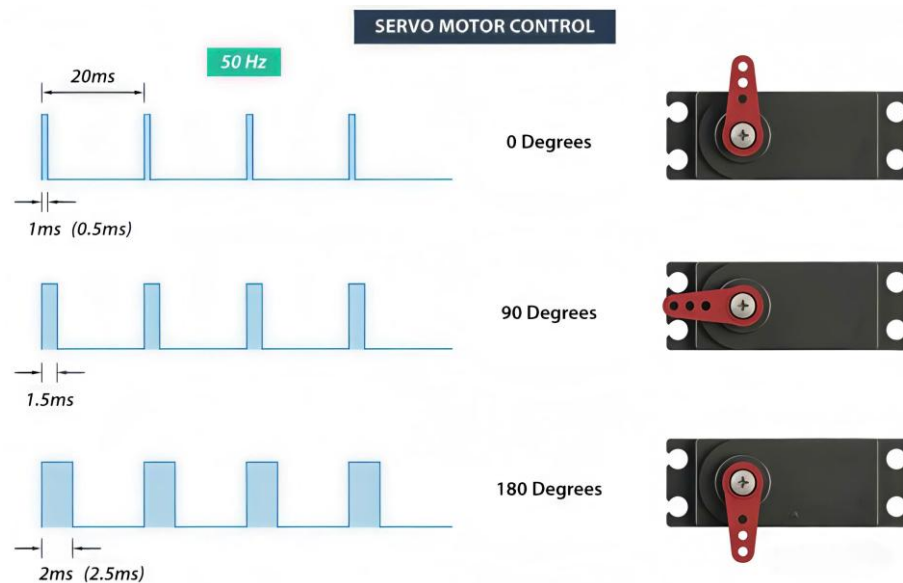


Fig 2.17: Servo motor control

Technical Specifications:

- Dimensions: 22.8 mm x 12.2 mm x 28.5 mm
- Weight: 13.4g
- Cable length: 175 mm
- Operating voltage: 4.8 - 6 V
- Operating speed: 0.1 sec/60 degrees (4.8 V); 0.08 sec/60 degrees (6 V)

- Maximum rotation angle: 90°

2.11. Module PCA9685



Fig 2.18: Module PCA9685

The PCA9685 is a 16-channel PWM controller IC with a 12-bit resolution that communicates via the I2C bus. It is commonly used in modules to expand the capability of controlling multiple servo motors or LEDs from a single microcontroller (such as Arduino, ESP32, or Raspberry Pi) while occupying only two signal pins (SDA and SCL).

Common Applications

Robotics Control: Highly popular for multi-joint robot projects such as spider robots or robotic arms that require the simultaneous control of multiple servo motors.

LED Control: Used to adjust the brightness (dimming) for RGBA LED systems or backlight displays.

Microcontroller Offloading: Thanks to its internal oscillator, the microcontroller does not need to continuously send signals to maintain the PWM pulses, thereby freeing up system resources.

Technical Specifications:

- Main IC: PCA9685
- Operating voltage: 2.3 - 5.5 VDC
- PWM channels: 16 channels, frequency: 40 - 1000 Hz
- PWM resolution: 12-bit
- Communication: I2C (Accepts 3 - 5 VDC TTL logic levels)
- Dimensions: 62.5 mm x 25.4 mm x 3 mm

Pin Connections:

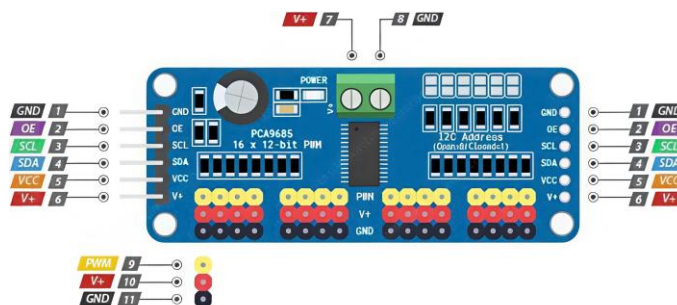


Fig 2.19: Module PCA 9685

- **VCC:** Power supply for the PCA9685 IC logic (typically 3.3V to 5V).
- **GND:** Ground pin.
- **SDA (Serial Data):** I2C serial data pin, connected to the microcontroller's SDA pin.
- **SCL (Serial Clock):** I2C serial clock pin, connected to the microcontroller's SCL pin.
- **OE (Output Enable):** Pulling it low enables the outputs; pulling it high disables all PWM outputs.
- **V+:** Dedicated power supply pin for the connected servo motors. Separating V+ and VCC protects the microcontroller from voltage drops and electrical noise during motor operation.
- **Output Pins:** 16 output channel headers (0-15) for direct motor connection. Each header consists of 3 corresponding pins: PWM (signal pulse), V+ (motor power), and GND.

2.12. Module RS232

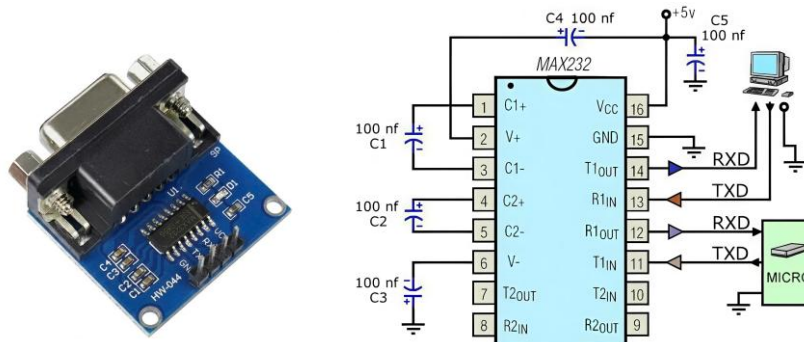


Fig 2.20: Module RS232 and Pinout

RS232 is used as a standard in order to link computer as well as its peripheral devices that allow the serial data exchange. The basis working of RS232 is to define the voltages for the path that is to be used for data exchange purposes among various devices. Signal level as well as common voltages, configuration of common pin and the control signals are specified. This standard was basically designed for modem and teletypewriter. Thus it don't provide support to several elements like framing of character, character encoding etc. Without providing support to such devices, this standard cannot be adopted to carry out data transfer. So, in order to overcome this issue, an integrated circuit known as UART i.e. "Universal Asynchronous Receiver Transmitter" is used with RS232.

The characteristics of the RS232:

- Logic level 1 has a voltage ranging from -3V to -12V and logic level 0 from +3V to +12V.
- The baud rate values: 9600, 19200, 28800, 38400, 56600, 115200 bps.

2.13. Hardware Design

2.13.1. Power Supply

Principle circuit

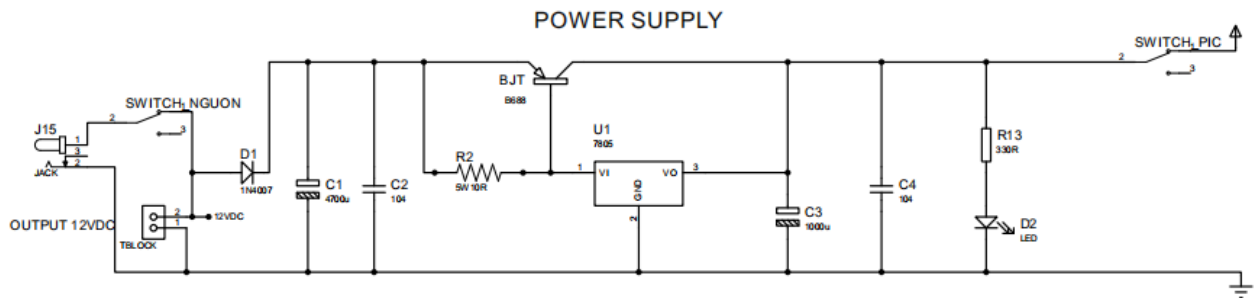


Fig 2.21: Power Supply

Components

1. IC LM7805

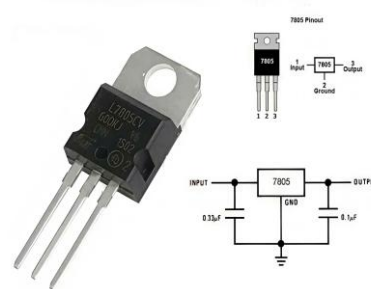


Fig 2.22: IC LM7805

IC7805 is a widely used integrated circuit (IC) in electronic applications for providing a stable power supply with an output voltage of 5V. It belongs to the 78xx family of linear regulators and was originally developed by National Semiconductor.

Specifications:

Input voltage: IC7805 requires a minimum input voltage that is at least 2V higher than the output voltage. Typically, input voltages in the range of 7V to 25V are used.

Output voltage: IC7805 provides a stable output voltage of 5V with high accuracy. This output voltage can be used to power other components and electronic circuits in the system.

Output current: IC7805 is capable of delivering a maximum output current of 1A. However, to ensure stability and optimal performance, it is recommended to limit the output current to a range of 500mA to 800mA.

Applications: IC7805 is commonly used in electronic applications such as programmers, power supplies for microcontrollers, and other components in electronic circuits. It can also be used in LED lighting applications, audio circuits, and various other applications requiring a stable 5V power supply.

2. BJT B688 PNP

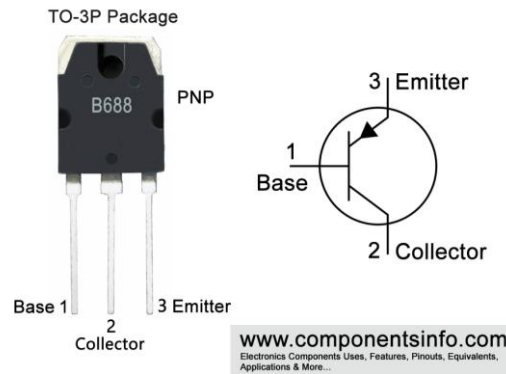


Fig 2.23: BJT B688 PNP

The B688 (commonly referred to by its full part number, 2SB688) is a high-power, silicon-based PNP (Positive-Negative-Positive) bipolar junction transistor (BJT). It is a widely utilized component in audio amplification systems and power switching circuits, primarily due to its robust capacity to handle high voltage and current levels.

Specifications of transistor B688:

- Collector - emitter voltage: -120 V
- Collector - base voltage: -120 V
- Generator - base voltage: -5 V
- Collector current: -8 A
- Radiator - 80 W
- DC current gain (hfe) - 55 to 160
- Forward frequency - 12 MHz
- Operating and storage junction temperature range -55 to +150°C

Pin Configuration:

With the printed face of the transistor facing towards you, the pin arrangement from left to right is as follows:

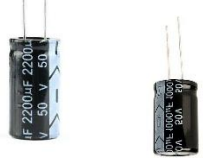
Pin 1 (B - Base): The control terminal.

Pin 2 (C - Collector): The collector terminal.

Pin 3 (E - Emitter): The emitter terminal.

3. Other Components

Num	Components	Quantity	Value	Images
1	Power resistor	1	5W10Ω	
2	Ceramic Capacitor	2	104pF	

3	Electrolytic Capacitor	2	2200uF, 1000uF	
4	Resistor	1	220Ω	
5	Diode 1N4007	1	X	
6	Switch	2	X	
7	Jack	1	X	
8	Led	2	X	
9	Domino	1	X	

2.13.2. Controller circuit

Principle circuit

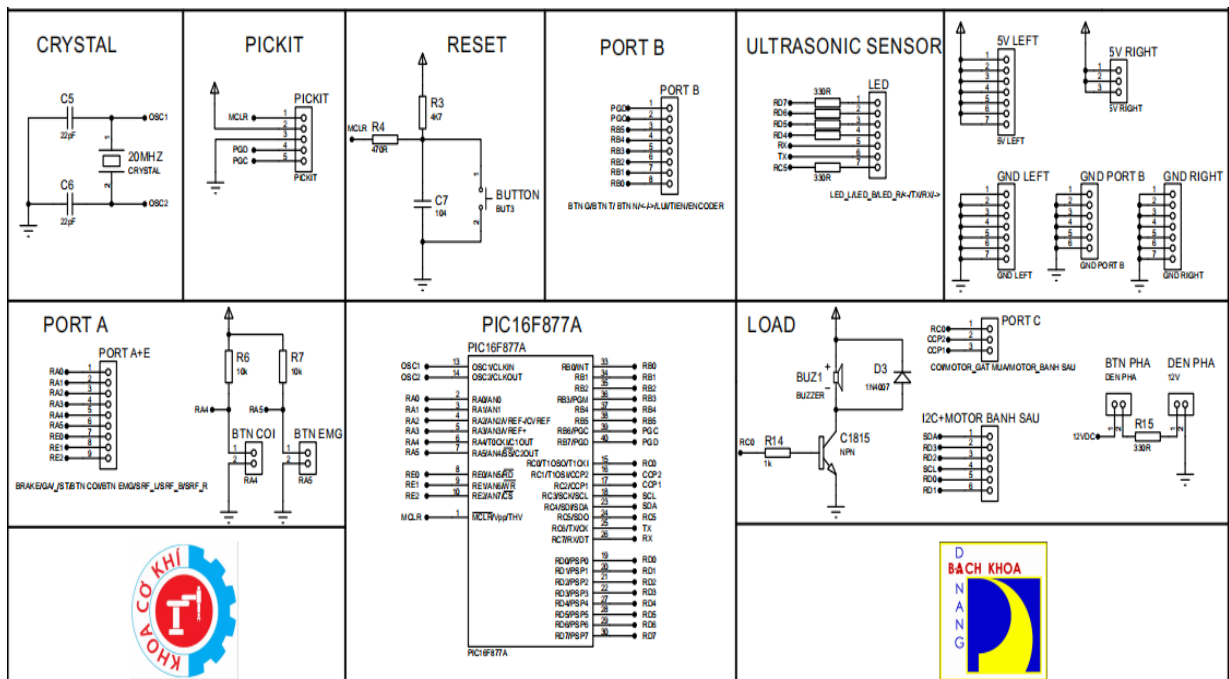


Fig 2.24: Controller Circuit

Components

Num	Components	Quantity	Value	Images
1	PIC16F877A	1	X	
2	Ceramic Capacitor	1 2	104pF 22pF	
3	Button 2pin	1	X	
4	Resistor	1 6 1 1	470 Ω 330 Ω 4.7k Ω 10k Ω	
5	40-pin DIP IC Socket	1	X	
6	Crystal oscillator	1	20Mhz	
7	Jump	12	X	
8	Buzze	1	5V	
9	BJT	1		
10	Diode 1N4007	1	X	

Actual Circuit

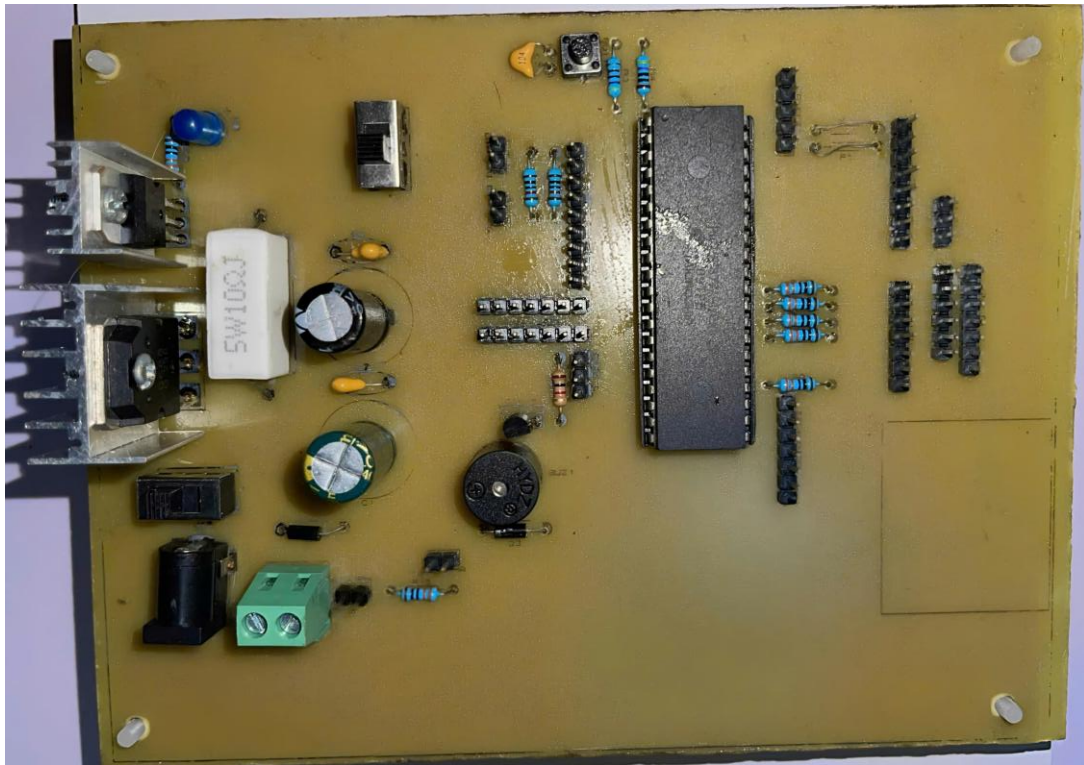


Fig 2.27: A photo of the Power Supply and Controller Circuit

Complete model:



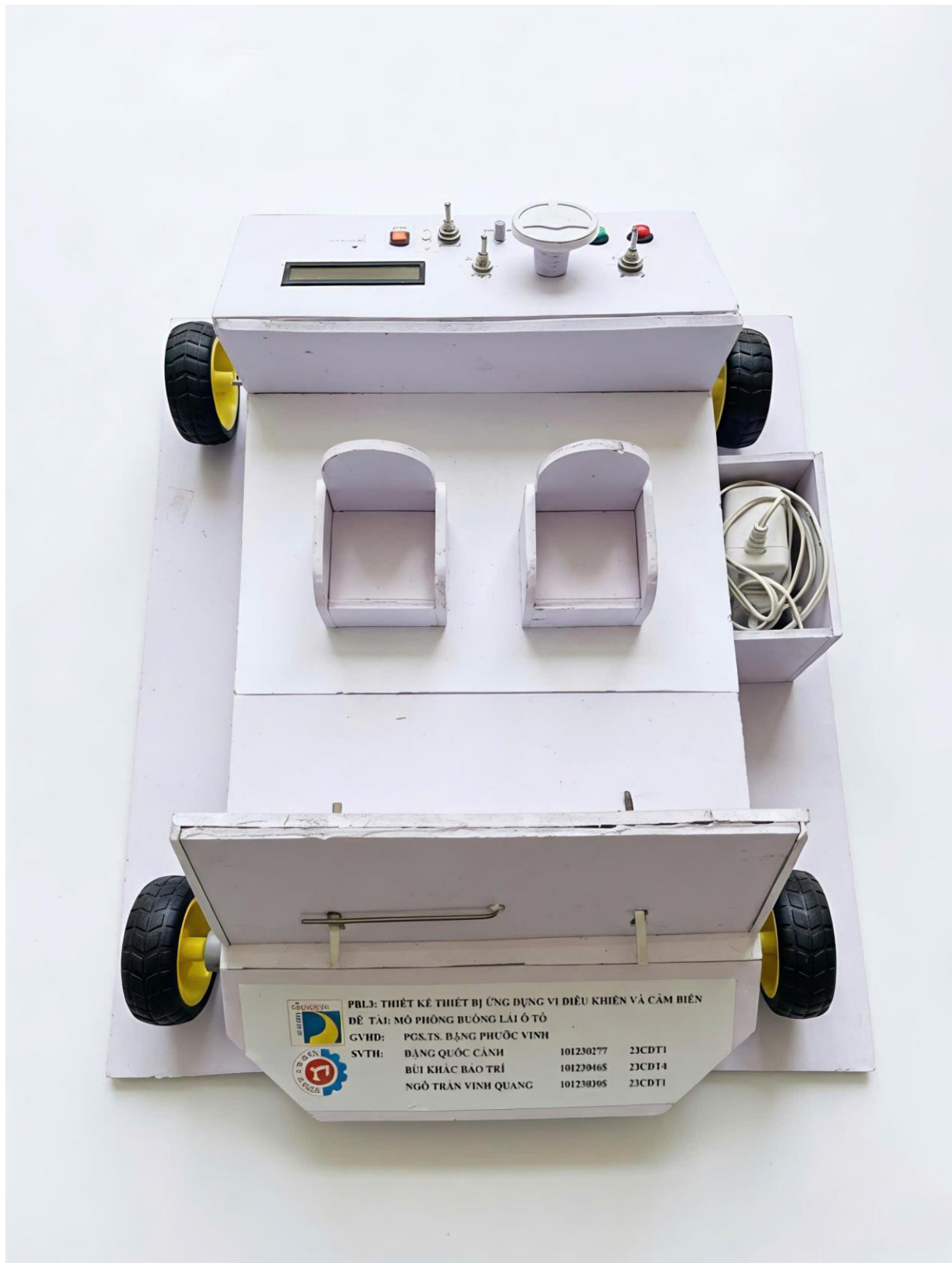


Fig 2.28: Complete Hardware Model

CHAPTER 3: SOFTWARE DESIGN

3.1. CCS Compiler



PIC C Compiler is an open-source source code compiler used for developing embedded applications on Microchip Technology's PIC (Peripheral Interface Controller) microcontrollers. Developed by CCS Inc., PIC C Compiler provides an integrated development environment (IDE) for writing and compiling C source code for various PIC microcontroller families.

PIC C Compiler supports a wide range of PIC microcontroller families, including PIC10, PIC12, PIC16, and PIC18. It provides the necessary tools and libraries to develop embedded applications on these microcontrollers.

With PIC C Compiler, developers can write source code in the C language, a high-level and readable programming language, while leveraging the features and power of PIC microcontrollers. The compiler will convert the C source code into corresponding machine code that the PIC microcontroller can understand and execute.

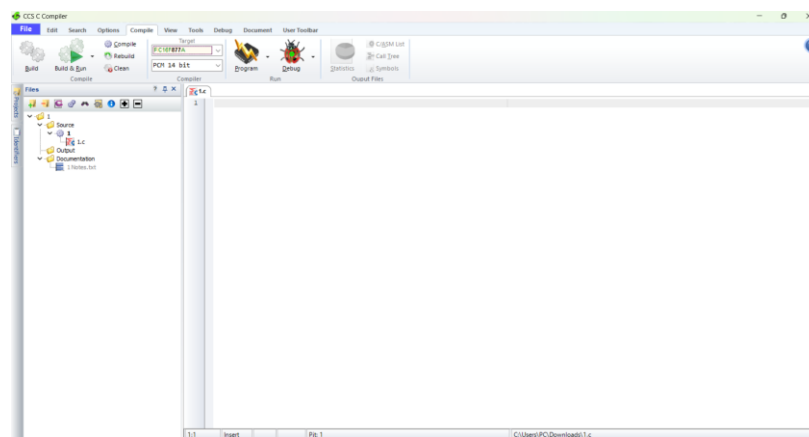


Fig 3.1: CCS C Compiler Interface

PIC C Compiler offers important features such as code optimization, support for standard libraries and manufacturer-specific libraries, debugging capabilities, and variable monitoring during runtime. It also supports integration with peripheral tools such as hardware debuggers and microcontroller simulation, making the development and testing process more convenient.

PIC C Compiler is a crucial tool in developing embedded applications on PIC microcontrollers. With it, developers can quickly and easily create reliable and high-performance embedded applications on Microchip's PIC platform.

3.2. PICKIT 3 v3.10

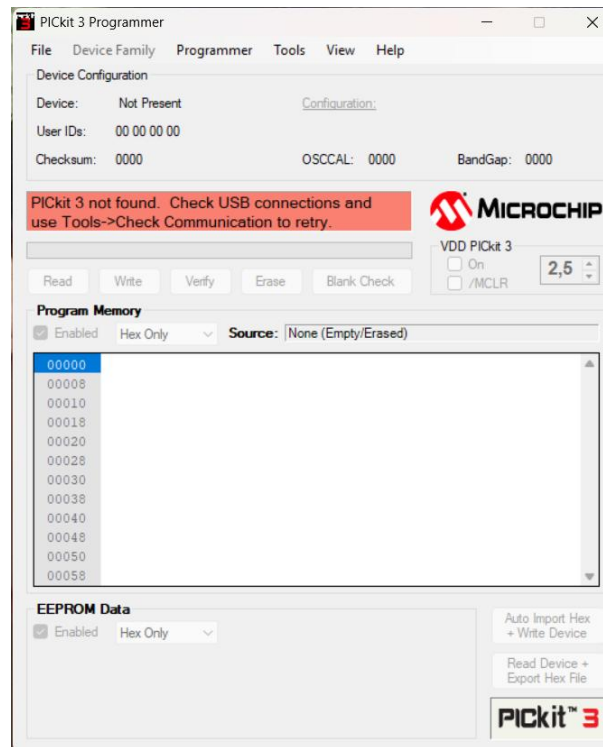


Fig 3.2: PICKIT 3 Interface

Pickit3 v3.10 is a software that supports programming and debugging for development applications on PIC (Peripheral Interface Controller) microcontrollers. It is the software counterpart of Pickit3, a popular programmer and debugger used for programming and loading code onto PIC microcontrollers.

Pickit3 v3.10 provides features and tools for programming code into PIC microcontrollers and debugging programs. Here are some key features of the Pickit3 v3.10 software:

- Programming PIC microcontrollers: The software allows you to program compiled source code into PIC microcontrollers through the USB interface of Pickit3. You can load code from HEX or ELF files generated from C compiler or other languages.
- Program debugging: Pickit3 v3.10 offers powerful debugging features to help you find and fix errors in your program. You can set breakpoints, monitor and inspect variable values, and trace the program execution flow.
- Support for multiple PIC microcontroller families: The Pickit3 v3.10 software supports various PIC microcontroller families, including PIC10, PIC12, PIC16, and PIC18. You can program and load code onto these microcontrollers by selecting the appropriate version and configuration within the software
- Integration with development environments: Pickit3 v3.10 integrates well with popular development environments such as Microchip's MPLAB X IDE. You can connect and use Pickit3 v3.10 for programming and debugging within your preferred development environment.

3.3. Algorithm Flowchart

3.3.1. Main program

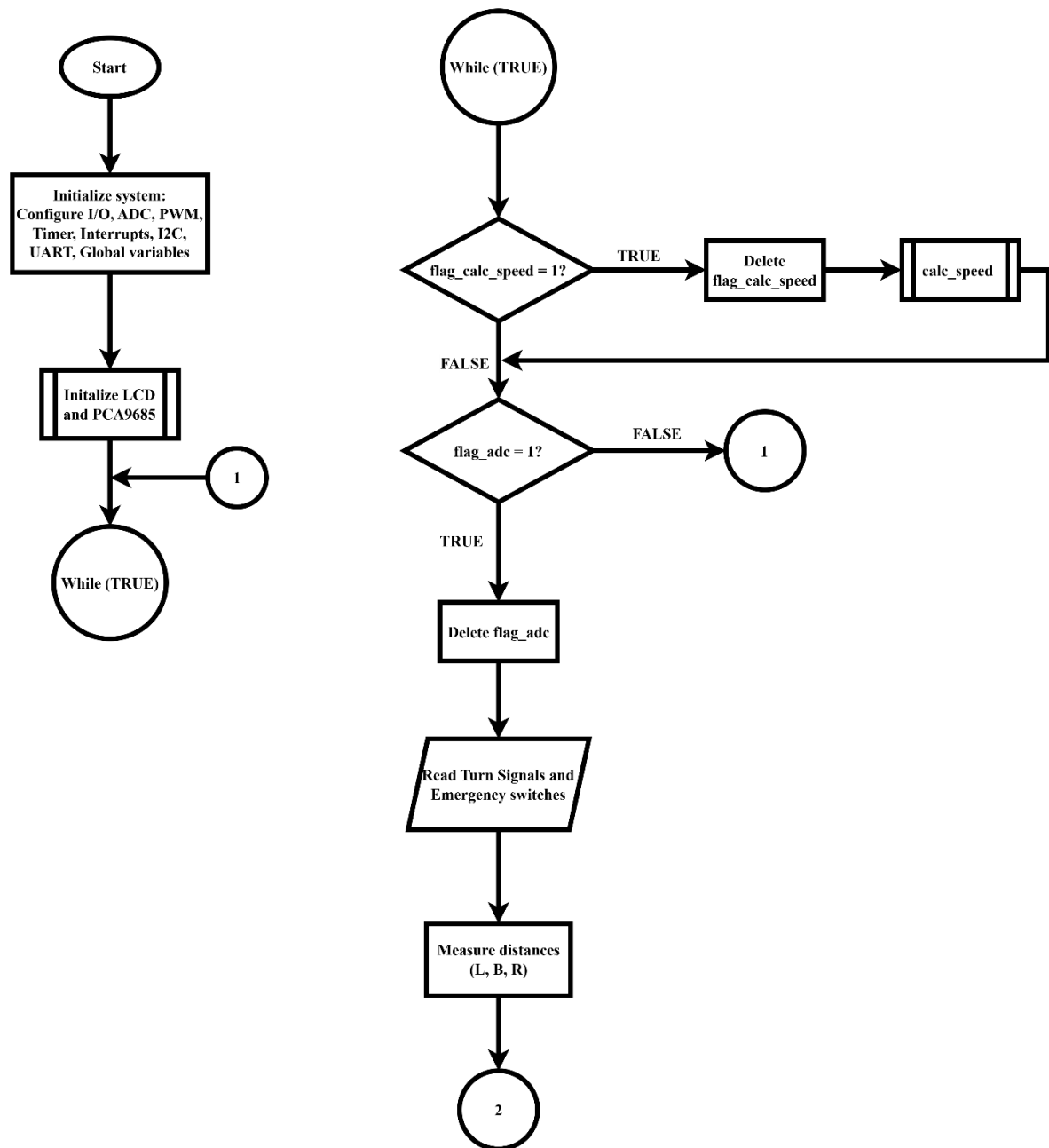


Fig 3.3: Main program 1

Figure 3.3 details the algorithm flowchart of the main program. It begins by initializing core hardware modules (I/O, ADC, PWM, UART) and peripheral devices (LCD, PCA9685). The microcontroller then enters an infinite, event-driven loop. This loop continuously polls specific interrupt flags: 'flag_calc_speed' (for velocity computation) and 'flag_adc' (to trigger switch readings and ultrasonic measurements).

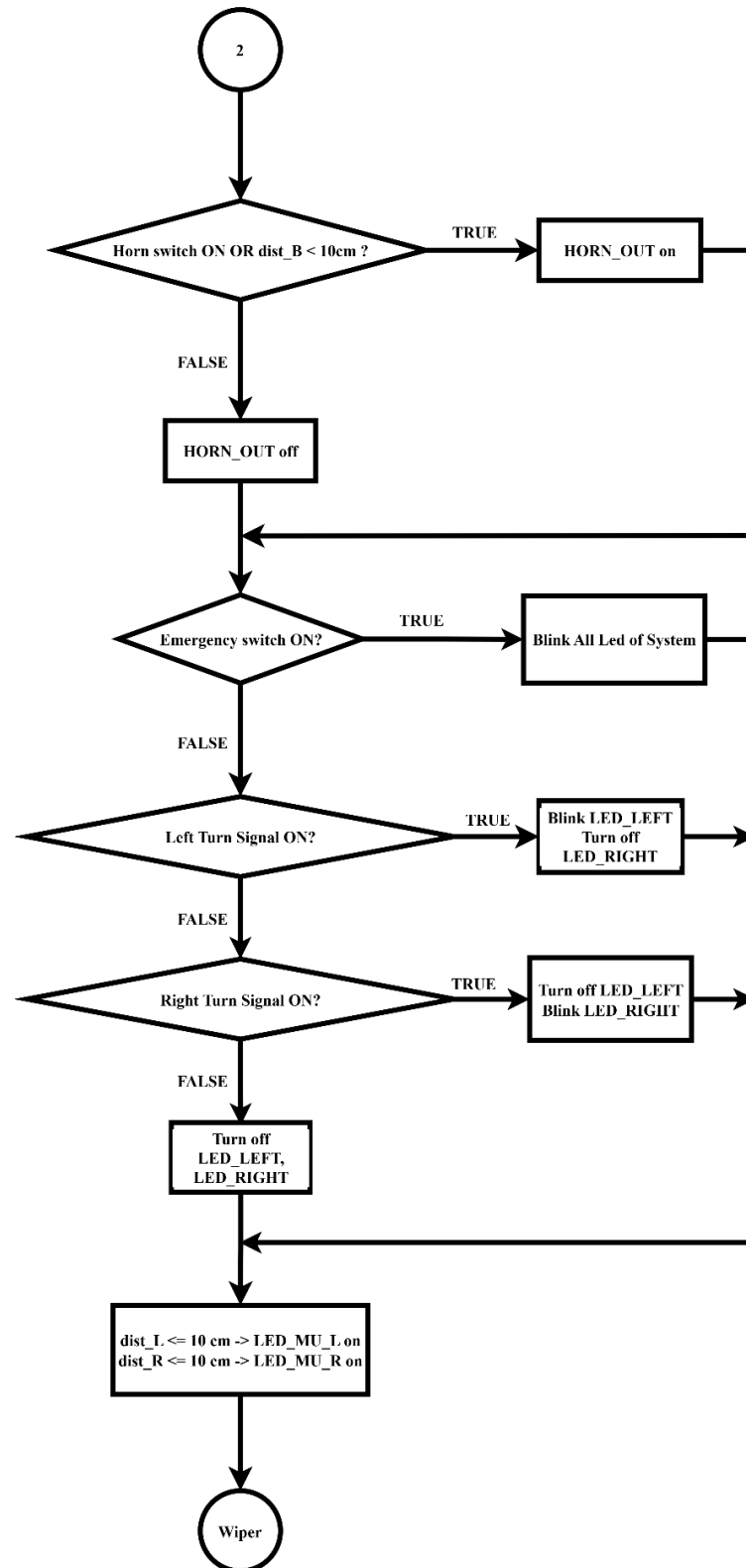


Fig 3.4: Main program 2

Figure 3.4 illustrates the control logic for the vehicle's acoustic and visual warning systems. It features active safety intervention by automatically triggering the horn if a rear obstacle is detected within 10cm ($\text{dist_B} < 10\text{cm}$), overriding driver input. The lighting control follows a strict priority hierarchy, where the emergency flasher switch supersedes individual turn signals. Finally, the system independently evaluates side ultrasonic data to activate blind-spot warning LEDs before transitioning to the wiper control module.

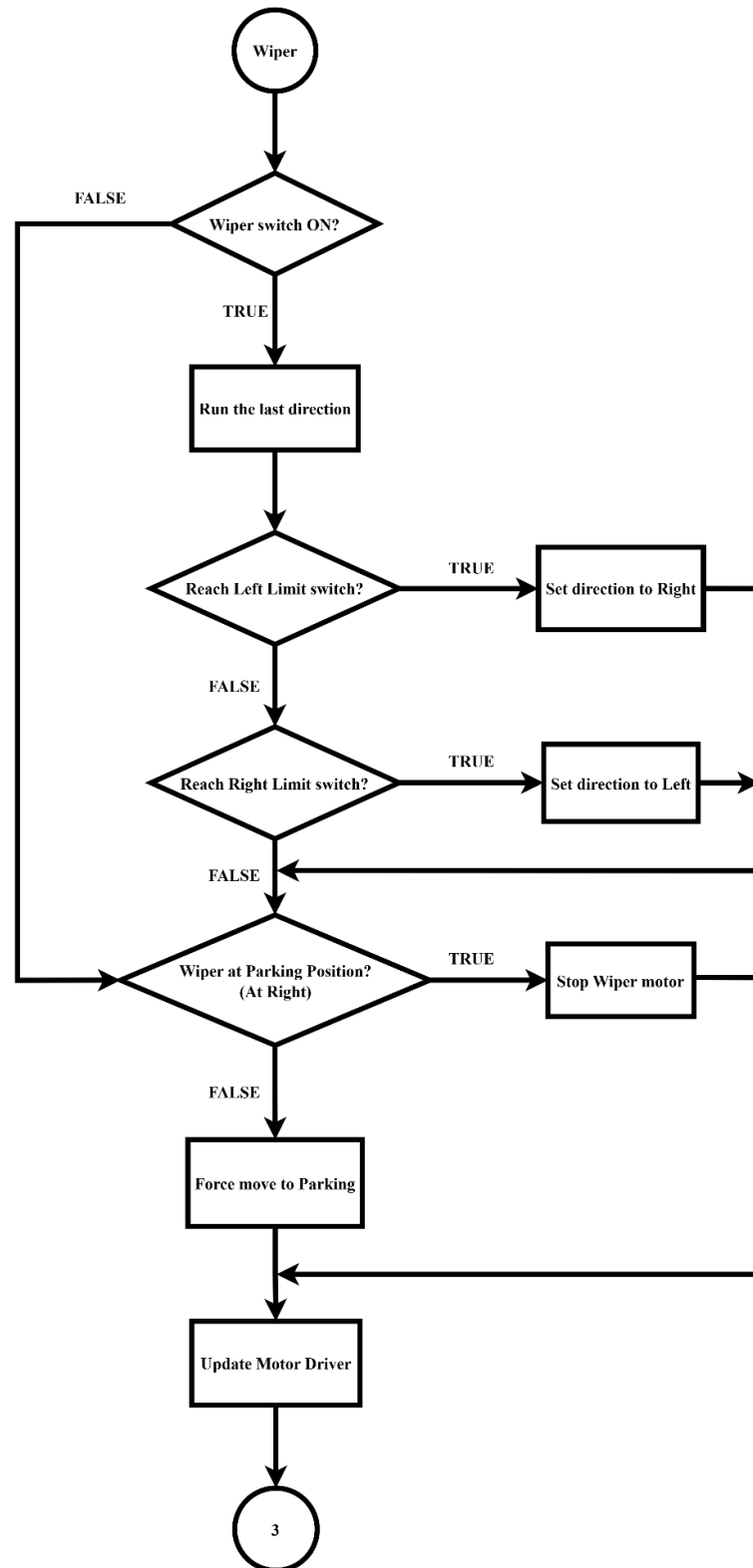


Fig 3.5: Main program 3

Figure 3.5 illustrates the wiper control algorithm featuring an intelligent Auto-parking mechanism. During active operation, the motor dynamically toggles its sweeping direction based on hardware feedback from the left and right limit switches. The core advantage of this logic lies in its deactivation sequence: when the switch is turned off, the system overrides the immediate power cut, forcing the motor to return the wiper blades to their designated parking position before coming to a complete stop.

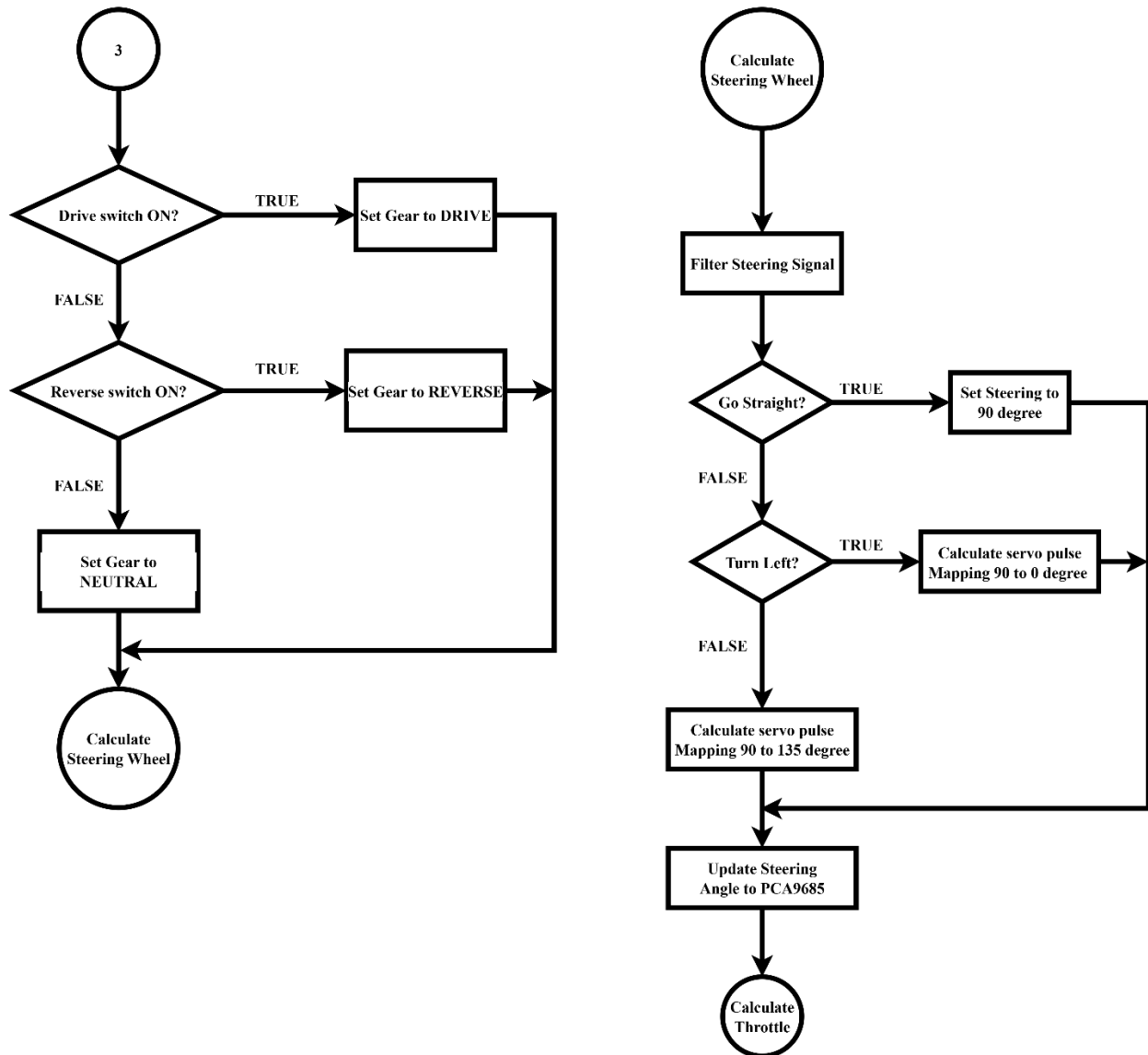


Fig 3.6: Main program 4

Figure 3.6 illustrates the flowcharts for gear state selection and steering angle calculation. The left section details the priority-based gear logic (Drive, Reverse, Neutral). The right section demonstrates the steering algorithm: raw input signals are first filtered, then passed through a middle zone check to enforce a straight driving state (90 degrees). Notably, the algorithm employs an asymmetrical angle mapping (90°->0° for left turns and 90°->135° for right turns) to strictly accommodate physical mechanical limits before updating the PCA9685 servo controller.

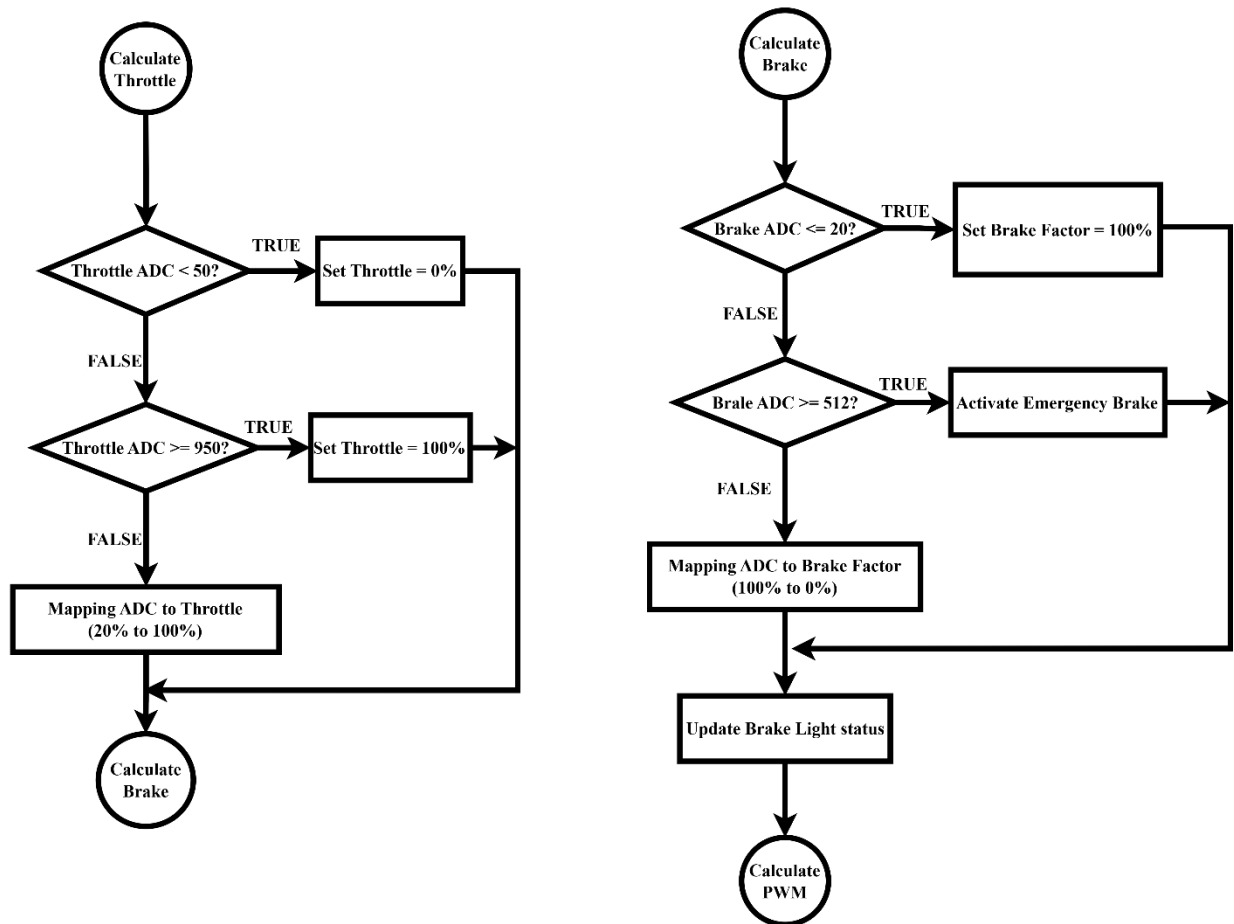


Fig 3.7: Main program 5

Figure 3.7 details the Throttle and Brake signal processing algorithms. Both logic flows incorporate 'dead zones' at their ADC boundaries to eliminate mechanical noise and jitter. Valid inputs are linearly mapped: the throttle translates to base power (20%-100%), while braking is evaluated as a dampening 'Brake Factor' (100% to 0%). The algorithm also features a hard-coded emergency brake override threshold to ensure maximum safety prior to the final PWM calculation.

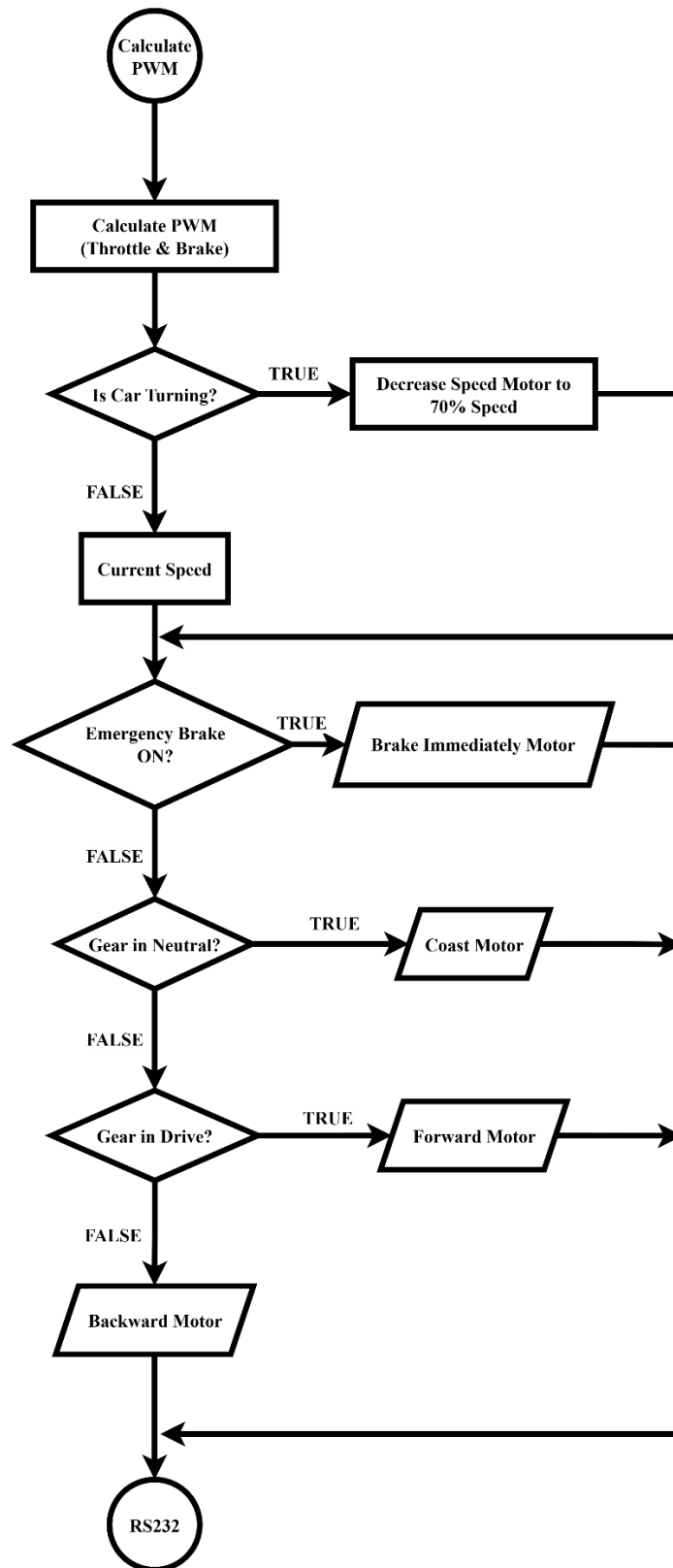


Fig 3.8: Main program 6

Figure 3.8 illustrates the PWM calculation and motor actuation flowchart. The algorithm features a 'Cornering Assist' mechanism, which automatically reduces motor power to 70% during turns to maintain vehicle stability. The final motor state is then determined through a strict priority-based logic chain: Emergency Braking holds the highest execution priority, followed by Neutral (coasting), Drive (forward), and finally Reverse.

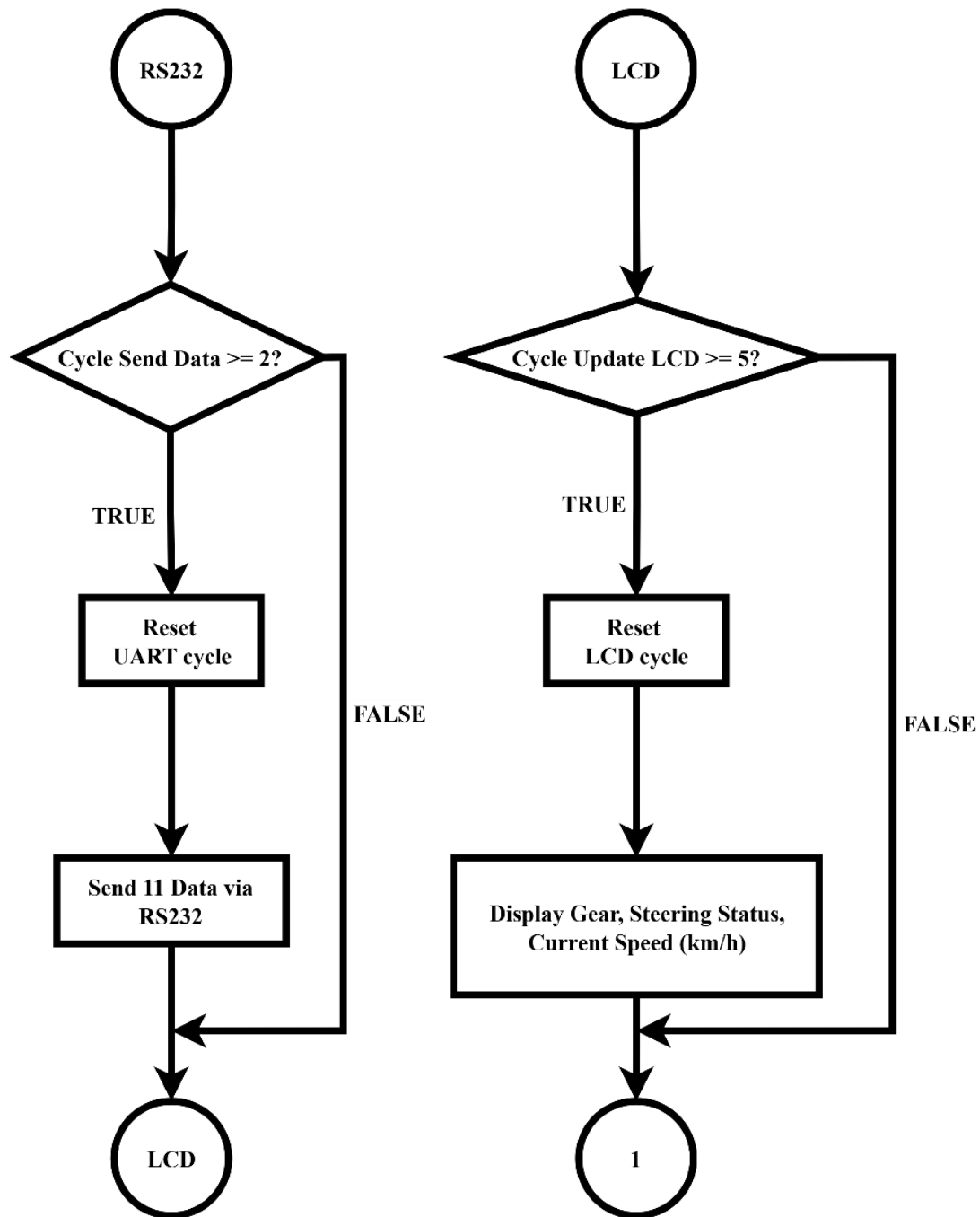


Fig 3.9: Main program 7

Figure 3.9 illustrates the flowcharts for the LCD interface and RS232 telemetry communication. To optimize bandwidth and prevent display flickering, the system implements independent cycle counters: 11 telemetry data points are bundled and transmitted via UART every 2 loop cycles, whereas the LCD updates gear, steering, and speed status every 5 cycles.

3.3.2. Function program

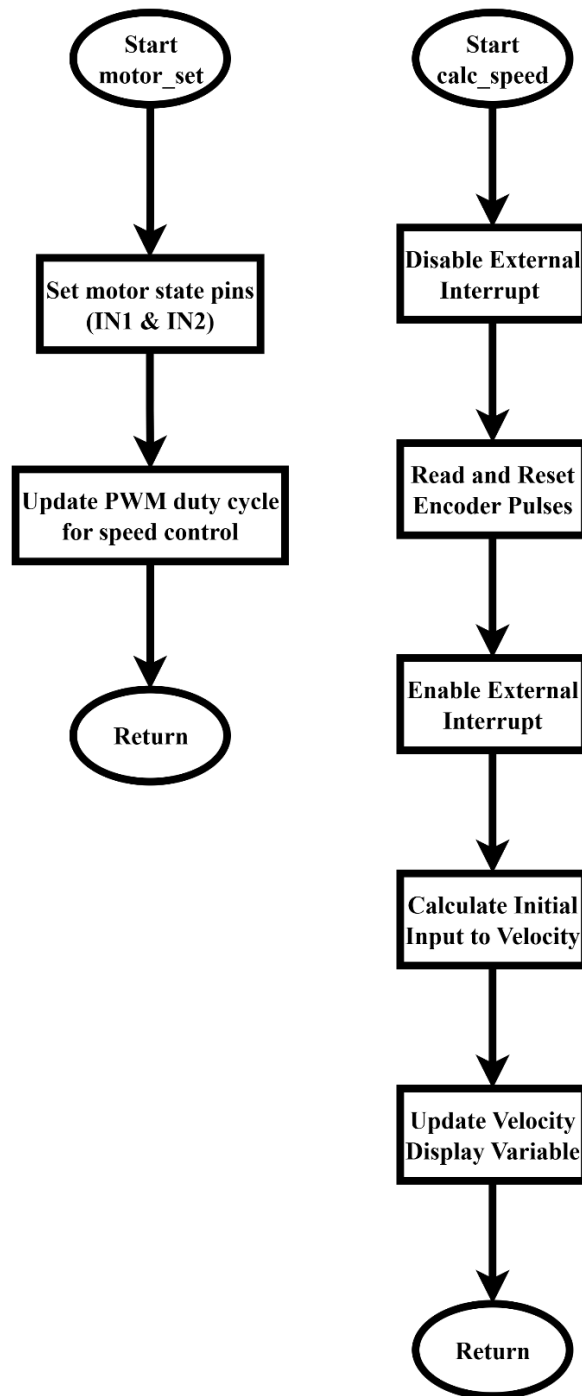


Fig 3.10: Function program 1

Figure 3.10 illustrates the 'motor_set' function configures the motor's direction pins (IN1 & IN2) and updates the PWM duty cycle to regulate speed. Meanwhile, the 'calc_speed' function calculates the vehicle's actual velocity from encoder pulses. It notably utilizes a "critical section" (temporarily disabling interrupts) to safely read and reset the pulse count, ensuring strict data integrity before updating the final speed variable.

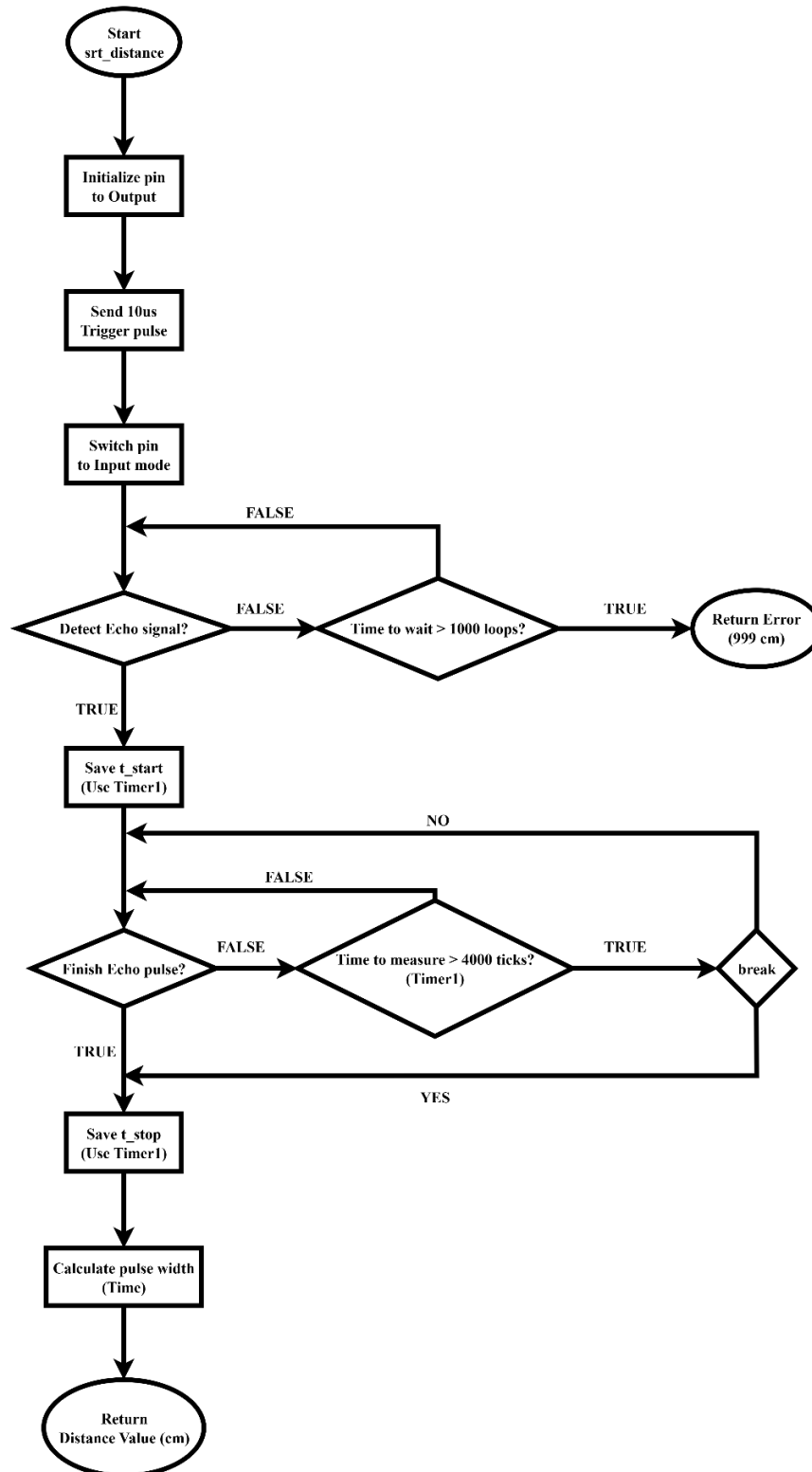


Fig 3.11: Function program 2

Figure 3.11 illustrates the 'srf_distance' function operates on the Time-of-Flight principle, sending a 10 μ s trigger pulse and utilizing Timer1 to measure the returning Echo pulse width. To prevent system freezing, the algorithm incorporates two critical timeout mechanisms: a 1000-loop software limit while waiting for the echo signal to initiate, and a 4000-tick hardware limit while waiting for it to finish. This ensures the system safely aborts and returns an error code (999 cm) if the sensor signal is lost, maintaining continuous operation.

3.3.3. External Interrupt program

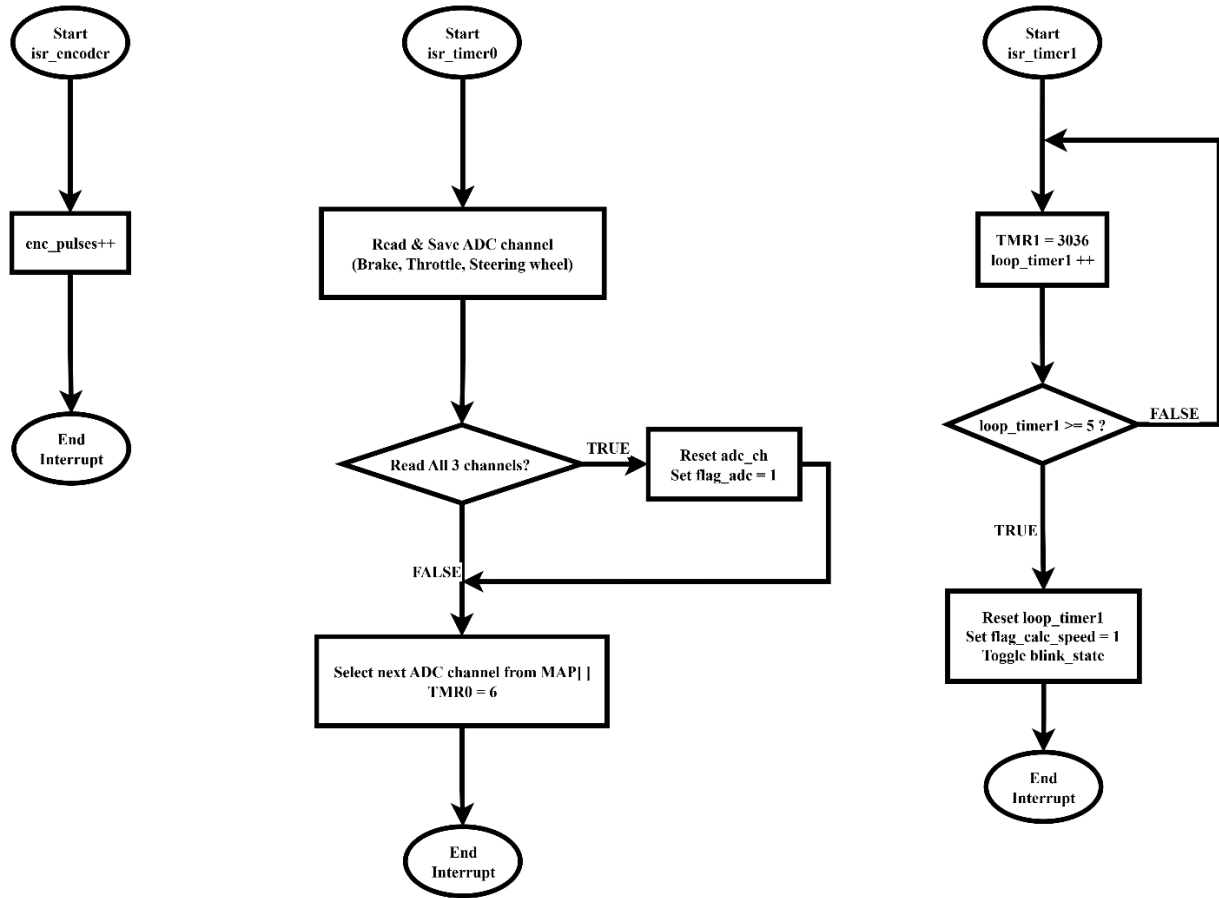


Fig 3.12: External Interrupt program

Figure 3.5 illustrates three critical Interrupt Service Routines for real-time multitasking: 'isr_encoder' function continuously counts wheel pulses in the background to prevent data loss; 'isr_timer0' function performs non-blocking, round-robin ADC scanning for user inputs; and 'isr_timer1' function acts as the system heartbeat, generating precise 0.5-second triggers for velocity calculations without using blocking delay functions.

3.4. Code CCS Compiler program

```

#include <16F877A.h>

#define ADC=10

#define fuses HS, NOWDT, NOPROTECT, NOLVP

#define use delay(clock=20MHz)

#define use i2c(Master, sda=PIN_C4, scl=PIN_C3, force_hw, fast)

#define use rs232(baud=115200, parity=N, xmit=PIN_C6, rcv=PIN_C7, bits=8, stream=PC)

#include "lcd_i2c.c"

#include "pca9685.c"
    
```

```
#use fast_io(A)
#use fast_io(B)
#use fast_io(C)
#use fast_io(D)
#use fast_io(E)

#define IN1  PIN_D0
#define IN2  PIN_D1
#define BTN_TIEN    PIN_B1
#define BTN_LUI     PIN_B2

#define SW_WIPER    PIN_B7
#define SW_LIMIT_1  PIN_B6
#define SW_LIMIT_2  PIN_B5
#define IN3         PIN_D2
#define IN4         PIN_D3

#define SRF_L       PIN_E0
#define SRF_B       PIN_E1
#define SRF_R       PIN_E2

#define SW_LEFT     PIN_B4
#define SW_RIGHT    PIN_B3
#define SW_HORN     PIN_A4
#define SW_EMG      PIN_A5

#define HORN_OUT    PIN_C0
#define LED_LEFT    PIN_D4
#define LED_RIGHT   PIN_C5
#define LED_MU_R    PIN_D5
#define BRAKE_LIGHT PIN_D6
#define LED_MU_L    PIN_D7
```

```
#define WHEEL_D_MM      65.0f
#define PULSES_PER_REV  20.0f

volatile int16 adc_raw[3];
volatile int8  adc_ch = 0;
volatile int1  flag_adc = 0;

volatile int32 enc_pulses = 0;
volatile int8  loop_timer1 = 0;
volatile int1  flag_calc_speed = 0;
volatile int1  blink_state = 0;

float  spd_display = 0.0f;

const int8 ADC_CH_MAP[3] = {0, 1, 3};

#int_TIMER0
void isr_timer0(void) {
    adc_raw[adc_ch] = read_adc();
    if (++adc_ch > 2) { adc_ch = 0; flag_adc = 1; }
    set_adc_channel(ADC_CH_MAP[adc_ch]);
    set_timer0(6);
}

#int_EXT
void isr_encoder(void) {
    enc_pulses++;
}

#int_TIMER1
void isr_timer1(void) {
    set_timer1(3036);
    if (++loop_timer1 >= 5) {
        loop_timer1 = 0;
    }
}
```

```
        flag_calc_speed = 1;
        blink_state = !blink_state;
    }
}

float srf_distance(int8 pin) {
    int16 t_start = 0, t_stop = 0, t_elapsed = 0;
    int16 timeout = 0;

    output_drive(pin);
    output_low(pin); delay_us(2);
    output_high(pin); delay_us(10);
    output_low(pin);
    output_float(pin);

    while (!input(pin)) {
        if (++timeout > 1000) return 999.0f;
    }

    t_start = get_timer1();
    while (input(pin)) {
        if ((get_timer1() - t_start) > 4000) break;
    }

    t_stop = get_timer1();
    t_elapsed = t_stop - t_start;

    return (float)t_elapsed * 0.0272f;
}

void calc_speed(void) {
    int32 p;
    float revs, circ_km, dist_km;
```

```
    disable_interrupts(INT_EXT);
    p = enc_pulses;
    enc_pulses = 0;
    enable_interrupts(INT_EXT);

    revs = (float)p / PULSES_PER_REV;
    circ_km = (3.14159f * WHEEL_D_MM) / 1000000.0f;
    dist_km = revs * circ_km;
    spd_display = dist_km * 7200.0f;
}

void motor_set(int1 m1, int1 m2, int16 pwm_ena) {
    output_bit(IN1, m1);
    output_bit(IN2, m2);
    set_pwm1_duty(pwm_ena);
}

void main(void) {
    int8  throttle = 0, brake = 100, gear = 0;
    int1  emg_brake = 0;
    int1  is_turning = 0;
    int16 servo_pulse = 1500;
    int16 full_pwm = 0, final_pwm = 0;
    int16 smooth_adc_steer = 512;
    int8  lcd_delay = 0;
    int8  rs232_delay = 0;

    float dist_L = 0.0f, dist_B = 0.0f, dist_R = 0.0f;

    int8 wiper_state = 0;
    int8 wiper_dir= 1;
    int1 is_wiper_on = 0;

    int1 sw_left_pressed = 0;
```

```
int1 sw_right_pressed = 0;
int1 sw_emg_pressed = 0;

set_tris_a(0xFF);
set_tris_b(0xFF);
set_tris_c(0x98);
set_tris_d(0x00);
set_tris_e(0x00);

port_b_pullups(TRUE);

output_low(IN1); output_low(IN2);
output_low(IN3); output_low(IN4);
output_low(LED_LEFT); output_low(LED_RIGHT);
output_low(LED_MU_L); output_low(LED_MU_R);
output_low(BRAKE_LIGHT);
output_low(HORN_OUT);

setup_adc_ports(AN0_AN1_AN3);
setup_adc(ADC_CLOCK_DIV_32);
set_adc_channel(ADC_CH_MAP[0]);
delay_us(20);

lcd_init();
lcd_goto(1,1); lcd_str((char*)"  ENGINE ON...  ");
delay_ms(1000); lcd_cmd(0x01);

pca9685_init();

setup_timer_2(T2_DIV_BY_16, 255, 1);
setup_ccp1(CCP_PWM);
set_pwm1_duty(0);

setup_ccp2(CCP_PWM);
```

```
set_pwm2_duty(307);

setup_timer_0(RTCC_INTERNAL | RTCC_DIV_32);
set_timer0(6);

setup_timer_1(T1_INTERNAL | T1_DIV_BY_8);
set_timer1(3036);

ext_int_edge(L_TO_H);

enable_interrupts(INT_EXT);
enable_interrupts(INT_TIMER1);
enable_interrupts(INT_TIMER0);
enable_interrupts(GLOBAL);

while (TRUE) {

    if (flag_calc_speed) {
        flag_calc_speed = 0;
        calc_speed();
    }

    if (flag_adc) {
        flag_adc = 0;

        sw_left_pressed  = !input(SW_LEFT);
        sw_right_pressed = !input(SW_RIGHT);
        sw_emg_pressed   = !input(SW_EMG);

        dist_L = srf_distance(SRF_L); delay_ms(10);
        dist_B = srf_distance(SRF_B); delay_ms(10);
        dist_R = srf_distance(SRF_R); delay_ms(10);

        if (!input(SW_HORN) || dist_B < 10.0f) output_high(HORN_OUT);
```

```
else                                                                 output_low(HORN_OUT);

if (sw_emg_pressed) {
    output_bit(LED_LEFT, blink_state);
    output_bit(LED_RIGHT, blink_state);
    output_bit(LED_MU_L, blink_state);
    output_bit(LED_MU_R, blink_state);
} else {
    if (sw_left_pressed && !sw_right_pressed) {
        output_bit(LED_LEFT, blink_state);
        output_low(LED_RIGHT);
    } else if (!sw_left_pressed && sw_right_pressed) {
        output_low(LED_LEFT);
        output_bit(LED_RIGHT, blink_state);
    } else {
        output_low(LED_LEFT);
        output_low(LED_RIGHT);
    }
    output_bit(LED_MU_L, (dist_L <= 10.0f));
    output_bit(LED_MU_R, (dist_R <= 10.0f));
}

if (!input(SW_WIPER)) is_wiper_on = 1;
else                  is_wiper_on = 0;

if (is_wiper_on) {
    if (wiper_state == 0) wiper_state = wiper_dir;
    if (!input(SW_LIMIT_1)) { wiper_state = 2; }
    else if (!input(SW_LIMIT_2)) { wiper_state = 1; }
} else {
    if (!input(SW_LIMIT_2)) { wiper_state = 0; wiper_dir = 1; }
    else if (wiper_state != 0) { wiper_state = 2; }
}

if (wiper_state == 1)      { output_low(IN3);
output_high(IN4); }
```

```
        else if (wiper_state == 2) { output_high(IN3); output_low(IN4);  
    }  
  
        else { output_low(IN3); output_low(IN4);  
    }  
  
        if (!input(BTN_TIEN)) gear = 1;  
        else if (!input(BTN_LUI)) gear = 2;  
        else gear = 0;  
  
        smooth_adc_steel = (int16)(((int32)smooth_adc_steel * 8 +  
adc_raw[2] * 2) / 10);  
  
        if (smooth_adc_steel >= 490 && smooth_adc_steel <= 530) {  
            servo_pulse = 1500;  
            is_turning = 0;  
        }  
        else if (smooth_adc_steel < 490) {  
            servo_pulse = 1500 - (int16)(((int32)(490 -  
smooth_adc_steel) * 1000) / 490);  
            is_turning = 1;  
        }  
        else {  
            servo_pulse = 1500 + (int16)(((int32)(smooth_adc_steel -  
530) * 500) / 493);  
            is_turning = 1;  
        }  
        pca9685_set_servo(0, servo_pulse);  
  
        if (adc_raw[1] < 50) throttle = 0;  
        else if (adc_raw[1] >= 950) throttle = 100;  
        else throttle = (int8)(20 + (((int32)(adc_raw[1] - 50) * 80)  
/ 900));  
  
        if (adc_raw[0] <= 20) { brake = 100; emg_brake = 0; }  
        else if (adc_raw[0] >= 512) { brake = 0; emg_brake = 1; }
```

```
        else { brake = (int8)(100 - ((int32)(adc_raw[0] - 20) * 100) /
492); emg_brake = 0; }

        output_bit(BRAKE_LIGHT, (emg_brake || brake < 80 || dist_B <
10.0f));

        full_pwm = (int16)((((int32)1023 * throttle * brake) / 10000);
        if (is_turning) final_pwm = (int16)((((int32)full_pwm * 7) /
10);

        else                final_pwm = full_pwm;

        if (emg_brake)      { motor_set(1, 1, 1023); }
        else if (gear == 0) { motor_set(0, 0, 0);    }
        else if (gear == 1) { motor_set(1, 0, final_pwm); }
        else if (gear == 2) { motor_set(0, 1, final_pwm); }

        if (++rs232_delay >= 2) {
            rs232_delay = 0;

            fprintf(PC, "%2.1f,%u,%u,%u,%u,%u,%u,%u,%u,%u,%u\r\n",
                spd_display,
                throttle,
                (sw_emg_pressed || (sw_left_pressed &&
!sw_right_pressed)) ? blink_state : 0,
                (sw_emg_pressed || (!sw_left_pressed &&
sw_right_pressed)) ? blink_state : 0,
                is_wiper_on,
                (gear == 1) ? 1 : 0,
                (gear == 2) ? 1 : 0,
                (dist_L <= 10.0f) ? 1 : 0,
                (dist_R <= 10.0f) ? 1 : 0,
                (dist_B <= 10.0f) ? 1 : 0,
                sw_emg_pressed
            );
        }
    }
```

```
        if (++lcd_delay >= 5) {
            lcd_delay = 0;
            lcd_goto(1, 1);
            if (smooth_adc_steer >= 490 && smooth_adc_steer <= 530) {
                if (gear == 1)      lcd_str((char*)"||  DRIVE  ||");
                else if (gear == 2) lcd_str((char*)"||  REVERSE ||");
                else                 lcd_str((char*)"||  NEUTRAL  ||");
            }
            else if (smooth_adc_steer < 490) {
                if (gear == 1)      lcd_str((char*)"<-  DRIVE      ");
                else if (gear == 2) lcd_str((char*)"<-  REVERSE    ");
                else                 lcd_str((char*)"<-  NEUTRAL    ");
            }
            else {
                if (gear == 1)      lcd_str((char*)"      DRIVE  ->");
                else if (gear == 2) lcd_str((char*)"      REVERSE ->");
                else                 lcd_str((char*)"      NEUTRAL ->");
            }
            lcd_goto(1, 2);
            lcd_str((char*)"V: "); lcd_float1(spd_display);
            lcd_str((char*)" km/h      ");
        }
    }
}
```

3.5. Visual Studio



Visual Studio is a powerful integrated development environment (IDE) developed by Microsoft. It provides a diverse set of tools and features for developing mobile, web, desktop, and cloud applications. Here is an overview of Visual Studio:

- Multi-language support: Visual Studio supports multiple programming languages such as C#, Visual Basic.NET, C++, F#, Python, JavaScript, and more. This allows developers to choose the language that best suits their project

- Integrated development tools: Visual Studio provides intelligent code creation tools and a robust code editor. It features IntelliSense, code suggestions, auto-completion, and code formatting, enhancing coding speed and reducing errors
- Debugging and testing: Visual Studio offers powerful debugging tools that allow developers to track and fix issues in their source code. It also provides various testing capabilities, including unit testing, functional testing, and automated testing
- User interface design: Visual Studio provides tools and UI designers for creating attractive and interactive user interfaces. You can create and edit interfaces using drag-and-drop functionality or build interfaces using code
- Collaboration and version control: Visual Studio integrates with version control tools such as Git and Azure DevOps (formerly Team Foundation Server), enabling team collaboration on projects and source code version management
- Mobile application development: Visual Studio supports mobile application development for iOS, Android, and Windows platforms. You can create cross platform mobile apps using Xamarin or build Windows Store and Windows Phone apps,...

Visual Studio is widely used by developers of all levels, from individual developers to large enterprise teams. It provides a feature-rich environment for efficient development, debugging, and collaboration, empowering developers to create high-quality applications across different platforms.

3.6. Visual Studio program

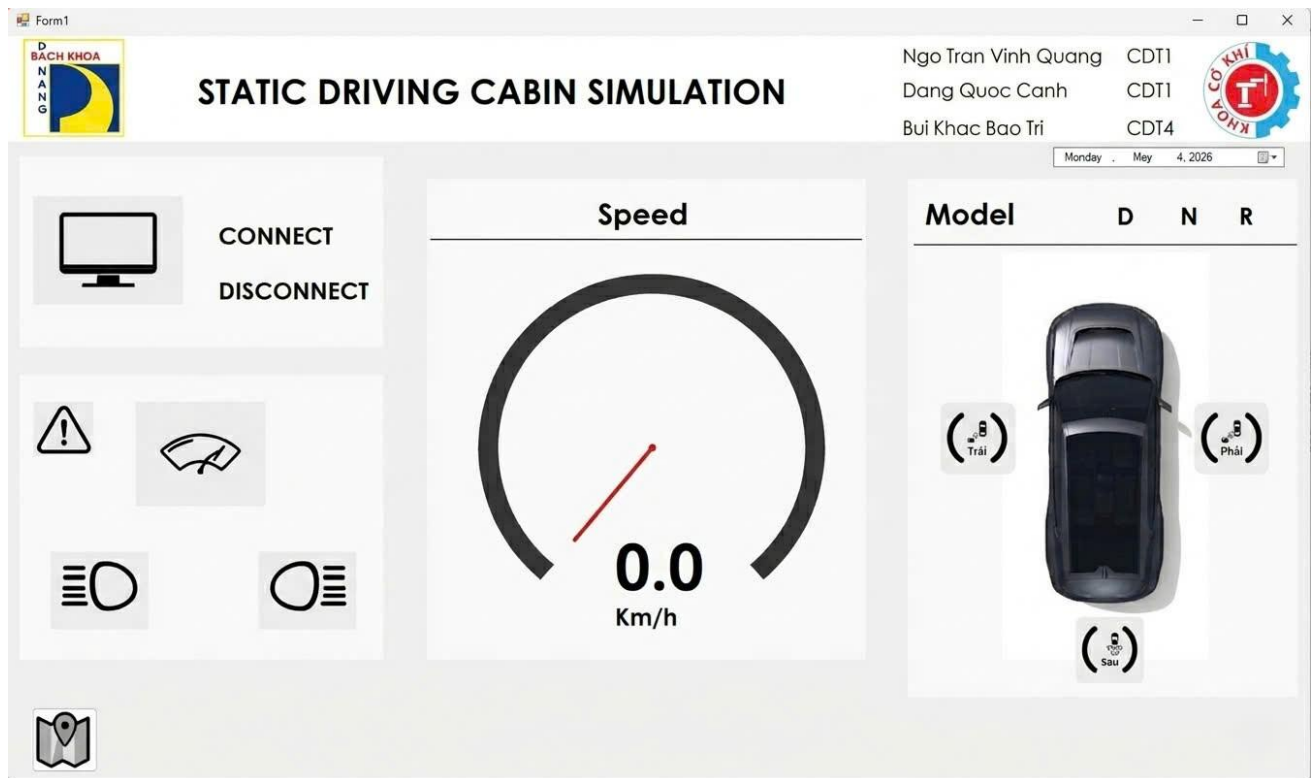


Fig 3.13: Dashboard

```
using Guna.UI2.WinForms;
using Microsoft.Web.WebView2.Core;
using System;
using System.Drawing;
using System.IO.Ports;
using System.Windows.Forms;

namespace pbl3
{
    public partial class Form1 : Form
    {
        private bool isMapReady = false;
        SerialPort serialPort1 = new SerialPort();

        public Form1()
        {
            InitializeComponent();
            InitSerialPort();
            InitMap();
        }
        private async void InitMap()
        {
            try
            {
                await mapView.EnsureCoreWebView2Async(null);
                isMapReady = true;

                mapView.CoreWebView2.Navigate("https://www.google.com/maps");
            }
            catch (Exception ex)
            {
                MessageBox.Show("Lỗi khi tải bản đồ: " + ex.Message);
            }
        }
    }
}
```

```
private void map_Click(object sender, EventArgs e)
{
    panelMap.Visible = !panelMap.Visible;
    if (panelMap.Visible)
    {
        panelMap.BringToFront();
    }
}
private void InitSerialPort()
{
    try
    {
        serialPort1.PortName = "COM3";
        serialPort1.BaudRate = 115200;

        serialPort1.DataReceived += new
SerialDataReceivedEventHandler(DataReceiveHandler);
        serialPort1.Open();
        connect.BackColor = Color.LimeGreen;
        connect.ForeColor = Color.White;
        disconnect.BackColor = Color.Transparent;
    }
    catch (Exception ex)
    {
        disconnect.BackColor = Color.Red;
        disconnect.ForeColor = Color.White;
        connect.BackColor = Color.Transparent;

        MessageBox.Show("Không thể mở cổng COM: " + ex.Message,
"Lỗi RS232", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void DataReceiveHandler(object sender,
SerialDataReceivedEventArgs e)
```

```
{
    SerialPort sp = (SerialPort)sender;
    try
    {
        string inData = sp.ReadLine();
        this.Invoke(new Action(() => ProcessData(inData)));
    }
    catch (Exception) {}
}

private void ProcessData(string data)
{
    try
    {
        data = data.Trim();
        string[] parts = data.Split(',');
        if (parts.Length >= 11)
        {
            hienthi.Text = parts[0];

            if (int.TryParse(parts[1], out int throttle))
            {
                if (throttle < tocdo.Minimum) throttle =
tocdo.Minimum;

                if (throttle > tocdo.Maximum) throttle =
tocdo.Maximum;

                tocdo.Value = throttle;
            }
            bool isLeftBlinking = (parts[2] == "1");
            bool isRightBlinking = (parts[3] == "1");
            if (isLeftBlinking)
            {
                xinhantraivang.Visible = true;
                xinhantraivang.BringToFront();
            }
            else
```

```
{  
    xinhantraivang.Visible = false;  
}  
if (isRightBlinking)  
{  
    xinhanphaivang.Visible = true;  
    xinhanphaivang.BringToFront();  
}  
else  
{  
    xinhanphaivang.Visible = false;  
}  
if (parts[10] == "1")  
{  
    canhbaodo.Visible = true;  
    canhbaodo.BringToFront();  
}  
else  
{  
    canhbaodo.Visible = false;  
}  
if (parts[4] == "1")  
{  
    gatmuavang.Visible = true;  
    gatmuavang.BringToFront();  
}  
else  
{  
    gatmuavang.Visible = false;  
}  
if (parts[5] == "1")  
{  
    tien.ForeColor = Color.Gold;  
    lui.ForeColor = Color.White;  
}
```

```
        mo.ForeColor = Color.White;
    }
    else if (parts[6] == "1")
    {
        lui.ForeColor = Color.Gold;
        tien.ForeColor = Color.White;
        mo.ForeColor = Color.White;
    }
    else
    {
        mo.ForeColor = Color.Gold;
        tien.ForeColor = Color.White;
        lui.ForeColor = Color.White;
    }
    if (parts[7] == "1")
    {
        mutraivang.Visible = true;
        mutraivang.BringToFront();
    }
    else
    {
        mutraivang.Visible = false;
    }
    if (parts[8] == "1")
    {
        muphaivang.Visible = true;
        muphaivang.BringToFront();
    }
    else
    {
        muphaivang.Visible = false;
    }
    if (parts[9] == "1")
    {
```

```
        musauvang.Visible = true;
        musauvang.BringToFront();
    }
    else
    {
        musauvang.Visible = false;
    }
}
}
catch (Exception ex)
{
    Console.WriteLine("Lỗi bóc tách: " + ex.Message);
}
}
private void Form1_FormClosing(object sender, FormClosingEventArgs
e)
{
    if (serialPort1 != null && serialPort1.IsOpen)
    {
        serialPort1.Close();
    }
}
}
```

CHAPTER 4: CONCLUSION

4.1. Results

- **Hardware Design:** Completed the schematic and optimized the Printed Circuit Board (PCB) layout, ensuring strict compliance with electronic engineering principles.
- **Fabrication and Assembly:** Directly executed the PCB manufacturing process (etching), soldered and assembled components, and successfully conducted testing to ensure stable hardware operation.
- **Model Construction:** Built and fully assembled a physical model simulating automobile mechanisms, including: front/rear wheel systems, steering wheel, accelerator and brake pedals, and auxiliary systems (windshield wipers, lighting/turn signals, etc.).
- **Control Programming:** Successfully programmed the PIC16F877A microcontroller in C language (using the CCS compiler). The control board seamlessly interacted with and operated all peripheral devices on the model.
- **Computer Communication:** Successfully established a data transmission protocol between the microcontroller and the PC via the RS232 standard, enabling the synchronization and real-time visual display of the cockpit dashboard interface on the screen.

4.2. Problems

- **Microcontroller Processing Limitations:** Although the PIC16F877A meets basic control requirements, its 8-bit architecture and limited number of I/O pins (40-pin package) prevent the system from managing all peripheral devices. A prime example is the headlight mechanism, which is currently not directly controlled by the microcontroller. Attempting to integrate too many functions easily leads to memory overload and the depletion of the chip's processing resources.
- **Mechanical Backlash:** Due to the manual assembly of the physical model, mechanical transmission components, such as the wheel axles and the steering mechanism, still exhibit a certain degree of mechanical backlash (play). This occasionally causes the input signals to lack the desired linearity and sensitivity during operation.

4.3. Future advantage

- **Processor Platform Upgrade:** Transitioning from an 8-bit architecture to higher-performance 32-bit microcontrollers (e.g., PIC32, STM32). This will fundamentally resolve memory overload issues and unlock the capacity to execute more complex control algorithms.
- **Intuitive 3D Simulation Interface:** Instead of being limited to a 2D dashboard display, data from the hardware model can be integrated with 3D track rendering software to create a comprehensive and highly interactive 3D virtual driving environment.

REFERENCES

- [1] Textbook “*Kỹ thuật vi điều khiển PIC*” by Assoc. Prof. Dr. Dang Phuoc Vinh and Assoc. Prof. Dr. Vo Nhu Thanh
- [2] <https://thanhvien.vn/voto-giai-phap-chat-luong-cho-cabin-mo-phong-dao-tao-lai-xe-185230223162659293.htm> (last accessed May 1, 2026)
- [3] https://vi.made-in-china.com/co_owatch/product_Exciting-360-Degree-6dof-F1-Car-Racing-Simulator-3-Screens-Car-Driving-Simulator-Car-Race-Game-Machine_yusenu Gug.html (last accessed May 1, 2026)
- [4] <https://fr.scribd.com/document/830607768/Introduction-to-Proteus> (last accessed May 4, 2026)
- [5] <https://hshop.vn/cam-bien-sieu-am-srf05> (last accessed May 4, 2026)
- [6] <https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf?fbclid> (last accessed May 4, 2026)
- [7] <https://dientutuonglai.com/fc-03-lm393-cam-bien-quang-encoder-chu-u.html> (last accessed May 4, 2026)
- [8] <https://robocon.vn/detail/ic33-ic-ne555-ic-tao-dao-dong.html> (last accessed May 8, 2026)
- [9] https://app.diagrams.net/#G1Y8TWghpoc598o7vpQX6ckN_M-tOdVthe (last accessed May 10, 2026)
- [10] https://www.az-delivery.de/fr/products/mg90s-micro-servomotor?srsId=AfmBOooDkKDHSO2IHkcvbzqchvQAK8JN5Ct8Xotl5dURnF_AcFGUx9P6 (last accessed May 10, 2026)
- [11] <https://hshop.vn/mach-xuat-16-xung-pwm giao-tiep-i2c> (last accessed May 10, 2026)